Research Master's Thesis

Static probabilistic timing analysis of worst-case execution time for random replacement caches

Author:
Bastien Pasdeloup

Supervisors:
Isabelle Puaut
Damien Hardy

Research team:
ALF

Abstract

In real-time systems such as those used in airbags, nuclear plants or space exploration, the correctness of the result of a program is as important as the time at which it is produced. Missing a deadline can have disastrous consequences such as the loss of human lives or ecological/financial catastrophes.

To face this problem, lots of techniques have been proposed to estimate the worst-case execution time of programs running in isolation (without considering the possible interactions with other programs running on the system). These estimates are then combined to ensure that real-time systems meet their timing requirements. Traditional deterministic methods return a single estimate of the worst-case execution time of the program under study. More recent techniques introduce probabilities, either to propose new ways to analyze programs, or to allow analyzing hardware with random behavior, such as random replacement caches.

After a bibliographic study presenting the advantages and drawbacks of the main methods used in worst-case execution time estimation, this document introduces a safe technique to estimate the probabilistic worst-case of a single-path program. The main idea of the technique is to separate the trace associated to the program under study into multiple smaller sub-traces on which independent analyses are performed using exhaustive cache states enumeration, and then to combine the results using convolution to obtain a distribution of probabilities for the possible execution times of the whole program. Experimental evaluation was performed on some single-path programs from the Mälardalen benchmark suite to study the precision of our method.

Acknowledgements

I first want to thank André Seznec for giving me the opportunity to realize this internship in ALF, and my advisors Isabelle Puaut and Damien Hardy for the constant help and support all along this period. Additionally, I want to thank everybody in the lab, and particularly Virginie Desroches who has helped me a lot with the administrative aspects.

Then, I want to thank the researchers from the BSC laboratory for providing me an effective library to compute convolutions.
I also want to thank Sebastian Altmeyer and Rob Davis for kindly allowing me to use their source code and thus being able to easily compare my results with theirs. Moreover, I want to thank them and Benjamin Lesage for providing me prompt and effective support with this code.

Finally, I want to thank my wife who accepted to read this work and point out some mistakes, eventhough static cache analysis sounds like Klingon language to her.

Contents

1 Introduction

In computer science, a program is said to be correct if it returns a correct result for every possible input. Real-time systems, such as those used in airbags or nuclear plant cooling systems, introduce a new dimension to the problem: the time it takes to compute the result. In this kind of systems, the time at which the result is produced is as important as the result itself. Missing a deadline might have disastrous consequences, such as human losses or ecological/financial catastrophes.

In order to face this problem, lots of techniques have been designed through the years to provide reliable information on the execution time of programs when running in isolation (without considering the possible interactions with other programs running on the system). The common aspect of all these techniques is that they aim at providing an estimate of the worst-case execution time (WCET) of the program under study. The estimates that are found for the various programs are then combined to allow the study of whole real-time systems. Therefore, to be trustworthy enough to be used in the validation process of real-time systems, the WCET estimate of a program must respect the following two criteria:

- *Safety:* the estimate is said to be *safe* if it is a hundred percent guaranteed to be greater or equal to all possible execution times of the program under study, including its WCET. If an estimate is proven to be safe, then the program under study is guaranteed to meet its deadline when running in isolation. For this reason, safety is essential for systems with a highly critical aspect. Such systems are called *hard real-time systems* in opposition to *soft real-time systems* in which safety is desired, but missing a deadline only degrades the quality of service, but does not jeopardize the system.
- *Tightness:* the tightness of an estimate is defined as its distance to the actual WCET of the program under study. An estimate is said to be *tight* if it is close to the actual WCET. When the schedulability of multiple real-time programs is studied, their estimates are used as building blocks. Therefore, if an estimate is not tight, more resources are needed to allow the programs to meet their timing requirements. As a consequence, tightness is determinant not to overestimate the hardware resources that are needed for running the whole system, thus minimizing the costs and the energy consumption. Tightness is very difficult to evaluate, since we do not know in general the actual WCET. It is generally estimated as in [1, 2] by using as a reference the result of a large number of simulations.

Figure 1: Basic notions in timing estimation of programs. The BCET and WCET represented here define the domain of all possible execution times, in which some measurements are made. In this example, the WCET estimate that is represented is safe since it is greater than the actual WCET.

Figure 1 illustrates the basic notions in timing estimation of real-time systems. It shows that measurements can be made to obtain possible execution times. However, there is in the general case no guarantee that any of the measurements covers a path leading to the WCET. For this reason, more advanced techniques are used. In Figure 1, the WCET estimate that is depicted is safe since it is greater than the maximal possible execution time.

Timing estimation techniques can be classified according to multiple criteria. Some of them emphasize safety, others do not. Some need precise information on the hardware platform to be used, others do not need it. In this document, the classification used is as follows:

- *Deterministic techniques:* deterministic timing estimation techniques are defined as methods that provide a *single* timing value as an estimate of the WCET of the program under study (this estimate may be safe or not).
- *Probabilistic techniques:* probabilistic timing estimation techniques are defined as methods that return *one or more* timing values, and associated probabilities. The meaning of the probabilities depends on the technique that is used, and is clarified later in the document[1].

Deterministic techniques have been studied for a long time, and most of them address a certain class of simple and deterministic hardware architectures. The probabilistic approaches appeared more recently, to provide estimation methods for programs running on another class of hardware: partially or fully randomized architectures. Additionally, because of the need for performance, current architectures are more and more complex and are therefore difficult to understand. Moreover, processor designers do not always provide a detailed description of their processors due to commercial protections. The probabilistic approaches aim at providing a first step towards the development of new timing estimation techniques for such architectures.

Among deterministic and probabilistic timing estimation techniques, the methods are distinguished according to the way they work. Two main approaches exist:

- *Measurement-based techniques:* measurement-based (or dynamic) timing estimation techniques are defined as methods that rely on measurements of the program under study to derive an estimate of its WCET. Such measurements can be made on code snippets or end-to-end, depending on the technique.
- *Static techniques:* static timing estimation techniques are defined as methods that do not execute the program under study to derive an estimate of its WCET. Instead, they use an abstract representation of the program and/or of the underlying hardware architecture.

This work introduces a safe probabilistatic static estimation method for single-path programs. It consists of a preprocessing allowing to separate the program into a partition of small windows on which exhaustive cache states analyses can be performed with a minimum loss of tightness. The results are then combined to provide a safe probabilistic WCET for the whole program.

Section 2 reviews the main classes of timing estimation methods. In addition to their requirements and to the way they work, their advantages and drawbacks are discussed. Section 3 introduces a new static probabilistic method to provide a probabilistic WCET for single-path programs. This method is then compared to the current state-of-the-art technique in Section 4.

[1]For a brief insight, the probabilities may represent for example the chance for a WCET estimate to be exceeded when executing the program. One might thus want these probabilities to be as small as possible.

2 Related work

In this section, existing techniques for estimating the WCET of programs taken in isolation are presented. Deterministic methods are first studied in Section 2.1. Then, probabilistic approaches are introduced in Section 2.2. For every method that is introduced[2], its utilization context is first given. Then, the method itself is presented. Finally, its advantages and drawbacks are discussed. A summary of the studied methods is provided in Section 2.3 to provide a concise vision of the classification used.

2.1 Deterministic techniques for estimating the worst-case execution time

This section presents the deterministic methods for estimating the WCET of real-time programs considered in isolation. These methods provide a single timing value as an estimate of the WCET of the program under study. Static methods are first presented in Section 2.1.1, and measurement-based methods are then described in Section 2.1.2.

2.1.1 Static deterministic timing analysis (SDTA) techniques

Static methods do not rely on executing the code of a program. Instead, such techniques study the possible executions of the program on an abstract model of the hardware architecture. Therefore, a detailed knowledge of the underlying architecture is required. Because they consider all possible executions of a program using an abstract representation of it, such methods provide a safe WCET estimate (provided that the model of the hardware that is used matches the actual target platform).

SDTA methods have two components. They include an analysis at the program level (*high-level*) to study the behavior of the program, and at the architecture level (*low-level*) to take into account hardware aspects such as cache memories.

2.1.1.1 High-level SDTA techniques

Timing estimation techniques at the program level operate on an abstract representation of the program under study. Several methods exist at this level. For the sake of conciseness, only the mostly used methods are presented here: one based on the code structure of the program (*tree-based computation*) and one using its control flow graph (*implicit path enumeration technique*).

Tree-based computation [22]

For this technique to be usable, one needs the abstract syntax tree (AST) of the program under study. Moreover, it is assumed that the program is correctly structured (correctly imbricated loops and `if` structures, as it is the case for most languages), and that the WCET $W(B_i)$ of every basic

[2]More precisely, the techniques that are introduced in this document represent categories of methods. To keep things simple, no distinction will be made between the methods and the categories they belong to.

block B_i is known. Finally, every loop must have a number of iterations bounded by a known integer k.

This method uses the AST of the program under study to derive an estimate of its WCET, by induction on the structure of the tree. The WCET estimate is computed using the following W function[3]:

$$\begin{cases} W(B_1;\ldots;B_n) & = & \sum_{i=1}^{n} W(B_i) \\ W(\texttt{if } T \texttt{ then } P_1 \texttt{ else } P_2) & = & W(T) + \max(W(P_1), W(P_2)) \\ W(\texttt{while } T \texttt{ do } P \texttt{ done}) & = & k * (W(T) + W(P)) + W(T) \end{cases}$$

The WCET of a sequence of basic blocks is equal to the sum of their individual WCET because they are executed sequentially. When there is a test, there are two possible paths that can be executed. Therefore, the WCET of a `if` structure is equal to the time that is needed to perform the test, added to the longest time among the two possible branches. Finally, the WCET of a loop is equal to the time it takes for one iteration, multiplied by the maximal number of times it can be executed, plus the additional test to exit the loop.

This method, in addition to being extremely simple, has the advantages to be scalable and to provide a good user feedback. Moreover, it returns a safe WCET estimate provided that the WCET estimates of the individual basic blocks are safe.

Because it works with the AST of a program, issued from its source code, the correctness of this method is affected by the aggressive compiler optimizations. Moreover, it can provide a bound that is not tight. This overestimation occurs because the method always assumes that the worst path is taken within a single control structure. As an example, consider the following program in which $\texttt{P}_1$ is long and $\texttt{P}_2$ is short:

```
i:=0; while (++i<100) do if (i%2=0) then P1 else P2 done
```

The technique considers that all iterations of the loop go through $\texttt{P}_1$ (because of the max in the induction function) even though there is an alternation between $\texttt{P}_1$ and $\texttt{P}_2$[4].

Implicit Path Enumeration Technique (IPET) [18]

For this technique to be usable, one needs the control flow graph (CFG) of the program under study. Moreover, it is assumed that the WCET $W(B_i)$ of every basic block B_i is known, and that every loop has a number of iterations bounded either by a known integer or by a linear combination of variables. Additionally, one can improve the tightness of the analysis by providing extra information (that can be represented as linear constraints). Examples of such possible information are mutually exclusive paths or relations between execution frequencies of basic blocks.

[3] More sophisticated versions of this function exist to allow the study of programs with more complicated structures. This simple function is chosen to illustrate the method.

[4] Here again, some more sophisticated methods use analyses to estimate which path is taken at which iteration, thus removing this problem for some simple tests.

This technique consists in writing constraints representing the program in order to solve a linear programming problem. The set of variables and constraints is described using the following rules (the numbers used in this enumeration correspond to the associated equations in Figure 2b):

1. Each basic block and edge is associated a constant time coefficient t_{entity} representing the contribution of the entity to the total execution time, and a variable execution counter x_{entity}. In the example presented in Figure 2b, the values of t_{entity} are equal to the times that are provided for the basic blocks.
2. Structural constraints are added to represent the possible flows in the program (edges in the control flow graph). In particular, the entry and the exit of the control flow graph must be taken once. The structural constraints force that the number of executions of a basic block is equal to the number of times it is entered, and to the number of times it is left. In Figure 2b, this is represented by the sums, in which the variables count the number of times the nodes and edges are visited.
3. Extra flow information is added to specify loop bounds (for example $x_a \leq 100$), mutually exclusive paths ($x_a + x_b = 1$), relations between execution frequencies of basic blocks ($x_a \leq 2 * x_b$), or any other useful information. In Figure 2b, the constraints specify that the loop has at most 100 iterations, and that B_3 is executed twice as much as B_4.
4. The objective function to maximize is the time spent executing the program. Solving the set of constraints instantiates the different x_{entity} so that the execution time is maximized, thus identifying a worst-case execution path.

(a)

(b)

1. $\begin{cases} \bullet\ \forall i \in [\![1;5]\!] : t_i = W(B_i); \\ \bullet\ t_{12} = t_{23} = t_{24} = t_{35} = t_{45} = t_{56} = t_{52} = 0; \end{cases}$

2. $\begin{cases} \bullet\ x_1 = x_6 = 1; \\ \bullet\ \forall b : x_b = \sum\limits_{a \in pred(b)} x_{ab} = \sum\limits_{c \in succ(b)} x_{bc}; \end{cases}$

3. $\begin{cases} \bullet\ x_2 \leq 100; \\ \bullet\ x_4 \leq 2 * x_3; \end{cases}$

4. `objective:` $\max \left(\sum\limits_{e \in entities} x_e * t_e \right)$

Figure 2: Example of definition of the set of variables and constraints (b) associated to the CFG representation of a program (a).

Since it identifies a worst-case execution path, IPET provides a safe WCET estimate for the studied program provided that the individual WCET of the basic blocks are safe. Moreover, because it works at the CFG level (that can be extracted from a binary file), it supports some aggressive compiler optimizations and can consider programs that do not need to be completely structured.

A reproach that can be made to this approach is that it is not as transparent for the user as the tree-based approach. However, tools such as GNU addr2line can provide more intelligible information to the user. Moreover, gathering information on execution frequency or loop bounds is not straightforward.

Thanks to its advantages, this approach has replaced the tree-based computation technique in most tools. It can be found for example in commercial/academic WCET estimation tools such as aiT, Bound-T, Heptane or SWEET [26].

2.1.1.2 Low-level SDTA techniques

The architecture level analysis is complementary to the high-level one. The aim of this analysis is to determine the behavior of the program under study on a given architecture abstract model. This low-level analysis is an important step because a lot of pessimism would be introduced if not considering for example cache memories (among other performance enhancing components).

Cache memories are small memories organized in layers to avoid fetching data or instructions from the main memory, thus accelerating the execution of the program. They have a structure defining where information should be stored in the cache: the main ones are *direct-mapped* (a program line has one possible location in the cache), *fully associative* (a program line can map to any cache line), or *set-associative* (a program line can map to a fixed number of locations in the cache). Additionally, cache memories have a replacement policy defining which entry in the cache should be evicted when a replacement is needed: the most classical ones are *LRU* (the least recently used line in the cache is evicted when space is needed), *FIFO* (lines are evicted in the order in which they are added), and *random* [24].

In this document, only one level of cache is considered. Other architectural elements such as caches hierarchies, pipelines or branch predictors are left aside. Research was made to address the analysis of architectures with such elements [14, 12, 7], but it is considered out of the scope of this work.

For a given program point, it is possible to know the contents of a cache memory by enumerating all possible cache states resulting from all possible paths leading to this point. They are called *concrete cache states* (CCS). However, this exhaustive approach clearly has an exponential complexity due to the number of possible paths in the control flow of the program, and can even be unfeasible if the paths cannot be enumerated. Another approach is to represent the cache contents in an abstract way: the *abstract cache state* (ACS).

Analysis using abstract interpretation [13]

This technique requires that the CFG of the program under study is available, and that the underlying architecture contains caches having a known structure and replacement policy. Moreover, the replacement policy that is used must have a notion of *aging* (for two memory references A and B, A is older than B if A was referenced before B). This is the case for example with the LRU replacement policy.

Abstract interpretation, formalized by Cousot and Cousot [8], is a semantics-based technique for

analyzing various aspects of a program. Here, it is used for the problem of predicting cache behaviors of a program (cache hits/misses). It was shown in [13] that the set of possible ACS form a complete lattice (one set of possible program lines per cache line, union operator). Therefore, a property on the cache contents that is valid in the *abstract world* of ACS is also valid in the *concrete world* of CCS. With this information, it is possible to classify the memory references (instructions or data) in the program according to their cache behavior.

In order to classify the memory references of the program, three analyses are performed. Each of them follows the control flow graph, and for every program point, performs two operations: *join* (when combining paths, describes the way the information is updated in the caches), and *update* (describes the modification of the cache contents according to the replacement policy). The analyses are defined as follows:

- *must analysis:* this analysis stores in the ACS the memory references that are *sure* to be in the cache, whatever path was taken before. It is done by performing the intersection of sets when joining paths, keeping the maximal age according to the replacement policy. This analysis allows to determine the memory references that always hit in the cache. An example for this analysis is depicted in Figure 3.
- *may analysis:* this analysis stores in the ACS the memory references that *might* be in the cache. It is done by performing the union of sets when joining paths, keeping the minimal age according to the replacement policy. This analysis allows to determine the memory references that always miss in the cache.
- *persistence analysis:* this analysis detects the memory references that *will remain* in the cache after being added for the first time. This allows to easily detect memory references that will first cause a miss and then will always hit. This is generally the case for memory references that appear in loops.

Figure 3 presents the application of the *update* and *join* functions when performing a *must analysis*. As an example, a fully associative instruction cache of 4 blocks with LRU replacement policy is considered[5]. Caches are represented as tables, with each column containing a set of possible memory references. The most recently used reference is depicted on the left, with the age of references increasing with the columns.

When the ACS are determined using the previously described analyses, every memory reference is associated one of the following categories:

- *always-hit* (AH): the memory reference is guaranteed to be in the cache. This is the case if it is in the ACS issued from the *must analysis*.
- *always-miss* (AM): the memory reference is guaranteed to always cause a miss. This is the case if it is not in the ACS issued from the *may analysis*.
- *first-miss* (FM): the memory reference will cause a miss at the first time, but is guaranteed to be in the cache afterward. This is the case if it is in the ACS issued from the *persistence analysis*.
- *not classified* (NC): in any other case.

[5]This technique can be used to study some other replacement policies, such as random [23, 1]. However, it leads to very imprecise results since we cannot know for sure what is evicted.

Figure 3: Example of application of the *update* and *join* functions when performing a *must analysis* on a fully associative instruction cache of 4 blocks with LRU replacement policy. Before `K` is executed, only memory references `A` and `E` are sure to appear in the cache.

This analysis has well-established mathematical foundations, and provides information about cache contents that allows the determination of a safe bound on the WCET of the program under study that is tighter than when not taking the hardware into consideration.

The main drawback of this approach is that the fixpoint iteration for computing the ACS can take a lot of time. Also, attention must be paid to the architecture when computing WCET estimate. As a matter of fact, the references classified as NC cannot be considered as AM in every architecture, since there exist timing anomalies in out-of-order processors [25].

Cache contents analysis based on abstract interpretation is implemented in WCET estimation tools such as aiT and Heptane [26].

2.1.2 Measurement-based deterministic timing analysis (MBDTA) techniques

Measurement-based methods rely on executing the code of a program on the target architecture or on a simulator. These techniques work in two steps: they first gather a chosen number of execution times, either for code blocks or end-to-end executions, and then use them to compute a WCET estimate of the whole program. For the case of code snippets measurements, they can be for example combined using the tree-based computation method that was presented in Section 2.1.1.1.

It is not possible in the general case to derive the exact WCET of a program using end-to-end measurements with random test cases, since there is no guarantee that a worst-case path is covered (enumerating all the paths is exponential in the general case). Therefore, there is a need for identifying properly such path. This can be done either by manually providing information on the input data that cause its execution, or by automatically deriving it. Since the former approach is human-based and thus error-prone, only the latter is presented here. Additionally, methods based on heuristics and genetic algorithms are introduced.

Automatic generation of tests for exhaustive paths coverage [27]

The only requirement for using this technique is that it must be possible to characterize the input domain of the program under study (*i.e* the possible values of variables that are used as input of the program).

This approach is based on the following observation: a same path in the control flow graph can be executed for distinct input values. With this observation, the following algorithm can be derived:

1. Choose an input value, and execute the program with it.
2. The corresponding path in the program is executed. Determine all input data that could have led to the same execution path. This is done by instrumenting the tests along the path, and using constraint solving to determine the input data that would have passed the same tests.
3. Start again until the input domain is fully covered.

A graphical view of the algorithm is depicted in Figure 4. Input data $d_0 \in D_0$ is first used to cause the execution of a path in the program. By analyzing the tests that were taken, the domain $D(d_0)$ causing the same execution path is determined. Then, input data $d_1 \in D_0 \setminus D(d_0)$ is used to find another path. This process continues until D_0 is fully covered.

Figure 4: Two iterations of the algorithm. Input data d_0 is used, and the associated domain $D(d_0)$ is determined. Then, $d_1 \in D_0 \setminus D(d_0)$ is selected, and $D(d_1)$ is determined.

Once the input domain is fully covered, it is guaranteed that the input data leading to a worst-case execution path are taken into consideration. For this method to be safe, the hardware must be in a state that leads to the WCET of the path that is executed. Determining the initial cache states leading to a worst-case execution is a very complex problem in the general case. However, for simple in-order architectures with caches implementing LRU replacement policy, empty caches can be used.

This method is implemented in the WCET estimation tool PathCrawler [27]. The tests that were performed by the authors of the tool suggest that the approach is efficient enough to scale up to large programs, provided that the number of feasible paths is limited.

Heuristics and genetic algorithms [26]

As for automatic generation of tests for exhaustive paths coverage, the requirement for using this technique is that it must be possible to characterize the input domain of the program under study.

The idea behind this MBDTA technique is to rely on heuristics and genetic algorithms to determine which inputs should be used for the end-to-end measurements. By performing carefully selected executions, the approach aims at covering a great quantity of the feasible paths in the program. Since this technique is extremely close to the previous one, it is not fully detailed here.

Because such methods are based on heuristics and genetic algorithms, they do not guarantee that all feasible paths are covered. Therefore, it is not possible to ascertain that a path leading to the WCET is covered. As a consequence, such methods are not safe.

A heuristics-based approach was implemented in a prototype of the TU Vienna real-time group [26].

2.2 Probabilistic techniques for estimating the worst-case execution time

Other approaches introduce probabilities to determine the WCET of programs. This choice is motivated either to cope with the sources of uncertainty related to program analysis or to take into account randomized hardware. Moreover, people from the PROARTIS [5] European project have the following philosophy: as hardware gets old, it has a probability of failure due for example to the aging of mechanical components. Vendors thus accept to provide platforms that have a probability of failure that is under a given (very low) threshold. In analogy to this, the objective of probabilistic approaches is to provide a WCET that has a (very low) probability of being exceeded (typically 10^{-15} per run in avionics [16]).

This section presents the probabilistic methods for studying real-time programs taken in isolation. Contrary to deterministic techniques, these methods provide one or more timing values, and associated probabilities of occurrence (from which exceedance probabilities can be derived). Since some notions such as safety and tightness are not straightforward in the case of probabilistic methods, some required background is presented in Section 2.2.1. Then, because they are the mostly studied up to now, the measurement-based probabilistic methods are first presented in Section 2.2.2. Finally, the static ones are studied in Section 2.2.3.

2.2.1 Required background for probabilistic estimation of WCET

In the case of deterministic estimation techniques, safety is defined as the certitude not to overestimate a given timing bound when running the program under study. However, the probabilistic methods cannot fully transpose this definition since they use probabilities in addition to timing estimates. In order to define this notion of safety for probabilistic WCET estimation methods, some other notions must be introduced.

2.2.1.1 Probabilistic execution time (pET) distribution

An probabilistic execution time (pET) distribution associates to every possible execution time of the program under study a probability of occurrence. This pET distribution is what is computed by the various probabilistic methods presented in this report. An example of such a pET distribution is given in Figure 5.

Figure 5: Example of pET distribution. Here, the studied program has nine possible execution times.

2.2.1.2 Cumulative distribution function (CDF), and its complement (CCDF)

From a pET distribution, a cumulative distribution function (CDF) can be built, by aggregating the probabilities in increasing order of time. Such a function represents the probability to be lower than a chosen time. Therefore, for a given time t, the probability associated to t in the CDF represents the probability not to exceed t when executing the program. The CDF associated to the pET distribution in Figure 5 is given in Figure 6 (top). The red line represents the probability to be lower than $t = 160$.

Figure 6: CDF (top) and CCDF (bottom) associated to the pET distribution in Figure 5. The red line represents the probabilities to be lower (respectively higher) than $t = 160$.

Equivalently, one can compute the complementary cumulative distribution function (CCDF) associated to the pET distribution, by computing for every pair $(t, \mathbb{P}(t))$ in the CDF (where $\mathbb{P}(t)$ is the probability associated to t) the pair $(t, 1 - \mathbb{P}(t))$. The CCDF associated to the pET distribution in Figure 5 is given in Figure 6 (bottom). Using the CCDF is more convenient for WCET estimation, since it represents for every execution time its probability to be exceeded.

2.2.1.3 Safety and tightness of a pET distribution

With the CCDF, it is now possible to define safety for probabilistic estimation methods. Since the CCDF represents the probability for every possible execution time to be exceeded, safety can be defined using the notion of order among pET distributions introduced in [19] as the following property:

Given the pET distribution E associating the real probabilities of occurrence for the possible execution times, and given an estimate $\hat{E}$ of E, we say that $\hat{E}$ is a safe estimate of E if for every random variables $X \sim E$ and $\hat{X} \sim \hat{E}$, for any timing value D: $\mathbb{P}(\hat{X} \geq D) \geq \mathbb{P}(X \geq D)$.

Graphically, the CCDF of a safe pET distribution estimate $\hat{E}$ must be over (or ideally equal to) the CCDF of the actual pET distribution E, as in Figure 7, not to underestimate the probability to exceed the execution time. In this figure, the green curve represents the CCDF associated to a pET distribution that estimates the one in Figure 5. The black one is the reference CCDF for this pET distribution taken from Figure 6. The line in red represents the timing value that must be taken to ensure that the estimate will be exceeded with a probability of $10/64$[6].

Figure 7: CCDF curves associated to the pET distribution in Figure 5 (black) and an estimate for it (green). The red line allows to determine the timing value associated to an exceedance probability of 8/64.

This figure also shows how tightness can be defined for pET distributions. Informally, the closer the estimated CCDF is from the actual one, the more precise is the estimation. However, obtaining the CCDF associated to the actual pET distribution is in general out of reach, since the number of paths (and thus possible execution times) in a program is generally exponential.

[6] One can thus determine the timing value to consider for any arbitrarily low probability of exceedance.

A solution to obtain the actual pET distribution is to simulate the program an infinite number of times. To be tractable, an approached pET distribution can be obtained by simulating the program a very high number of times. Evaluating the tightness of an estimated pET distribution for a given program is therefore generally done by comparing its associated CCDF to this simulated CCDF.

2.2.2 Measurement-based probabilistic timing analysis (MBPTA) techniques

Approaches based on measurements are the most developed among probabilistic techniques. As a matter of fact, probabilistic approaches form the core of the European project PROARTIS [5] in the form of MBPTA techniques.

MBPTA techniques compute a pET distribution using measurements. From the complementary cumulative distribution function (CCDF) associated to this pET distribution, a pair <timing value, probability of exceedance> can be extracted for an arbitrarily low probability. It is denoted as pWCET. Two approaches for MBPTA are presented here: one based on extreme value theory, and one computing inductively the probabilities to execute the possible paths.

Extreme value theory (EVT)-based approach [9]

For this technique to be usable, an important requirement has to be fulfilled. The observations that are issued from the measurements must be *independent and identically distributed* (i.i.d.)[7]. Having a system behaving as a random process (including the hardware and the operating system) fulfills this requirement. For example, in [15], a cache design with random placement and replacement policies is proposed.

Central limit theory is a mathematical theory that studies the average of a distribution, when repeating an experiment. EVT can be seen as the counterpart of this method, and studies the behavior of the tail of the distribution. From a set of observations issued from a random variable, EVT estimates the probability of *extreme values* to occur. In the case of real-time systems, observations are end-to-end timing measurements, and extreme values represent the WCET estimates. Multiple approaches exist for EVT. One of them, *Block Maxima Model* (BMM), considers the largest observations obtained in blocks of samples. This approach is very interesting for the following reason: it was shown that with this sampling process, the extreme values follow a known distribution of probabilities (Gumbel, Frechet or Weibull) [6].

The idea here is to collect execution times using measurements. Then, using these values, a model of the probabilities of occurrence of the extreme values is derived using EVT with BMM sampling. Finally, to compute the time t associated to a given probability p of exceedance (*i.e*, the pWCET $<t, p>$), the CCDF is used.

A synthetical view of the approach is depicted in Figure 8. Measurements of execution times have been made, and are presented according to their observed number of occurrence. Using some of them (chosen with BMM), EVT computes a distribution of the tail (in red). Then, the time associated to a given probability of exceeding the actual WCET is found. In this example are presented the

[7] More precisely, it has been shown [21, 17] that a linear stationarity is sufficient.

pWCET $<\alpha, 10^{-15}>$ and $<\beta, 10^{-50}>$.

Figure 8: Computation of the timing values α and β associated to the probabilities 10^{-15} and 10^{-50} of being exceeded using EVT. The distribution of probabilities associated to the extreme values, derived from the execution time measurements, is represented in red.

This method uses probabilities to take into account the uncertainty that is related to the inputs of the program. It constitutes a technique for deriving a pWCET for a program, for an arbitrary probability of being exceeded, and is therefore suitable for systems that can tolerate errors occurring under a given probability, such as soft real-time systems.

The pWCET obtained with this method is not safe because EVT computes a pWCET representative of the sampled execution times (that might not include a worst-case path).

Because this approach is quite new, to our best knowledge, no publicly available tool exists that implements it.

Paths execution probabilities computation [4]

For this technique to be usable, one needs the AST of the structured program under study. Therefore, the program must be well-structured.

In this technique, measurements are used to determine execution time profiles of code snippets. They are then combined using a tree-based approach, as presented in Section 2.1.1.1. Combination of execution profiles x and y associated to a sequence of basic blocks `X` and `Y` is made using a convolution operator defined as follows: $(x \otimes y)(t) = \sum_{\forall s} x(s)y(t-s)$. This operator replaces the addition in the tree-based computation presented in Section 2.1.1.1. A technique for replacing the multiplication and the maximum operators (used for the loops and `if` structures) is presented in [4]. Figure 9 presents the computation of $(x \otimes y)(5)$.

A problem that can be seen with this approach is the exponential growth in the number of distinct possible execution times. To cope with this problem, authors of [20] use a WCET-adaptated version of statistical *re-sampling* [3], consisting in reducing the number of possible execution times by

Figure 9: Convolution of execution time profiles x and y associated to basic blocks `X` and `Y`. For example, $(x \otimes y)(5) = x(1)y(4) + x(2)y(3) + x(3)y(2)$.

grouping them together. Here, it is done in a conservative way not to underestimate the probability of occurrence of the highest possible time. A simple re-sampling technique is depicted in Figure 10. It keeps the probabilities exceeding a certain threshold (here, 0.2) and the one associated to the highest execution time. Then, every time for which the probability is not kept is grouped with the kept one that is higher. More sophisticated approaches exist, and are presented in [20].

Figure 10: A simple re-sampling technique. Execution times with a probability over a threshold of 0.2 are kept, and others are added to the kept one that is just higher. The repartition of the removed probabilities is presented on the left, and the new execution time profile is on the right.

This method uses probabilities to take into account the uncertainty that is related to the underlying hardware, since measurements cover all the code in form of snippets. Since this approach can determine all possible execution paths (including the ones leading to the actual WCET), and associates to them their probabilities of occurrence, it can provide a safe pWCET using the CCDF.

However, since the execution time profiles of code snippets are computed using measurements, this method is safe if and only if all execution time profiles include the WCET of the code segment that is measured. Another drawback of this method is that it requires an effective re-sampling technique to reduce the exploding number of possible execution times.

Such an approach is implemented in the WCET estimation tool RapiTime [26].

2.2.3 Static probabilistic timing analysis (SPTA) techniques

The philosophy of SPTA is different from the one of measurement-based approaches. As it was shown before, the latter rely on measurements and derive a pWCET estimation using either EVT or tree-based methods. The former refuses the assumption that the paths leading to the actual WCET are captured by measurements only, and enforces their consideration.

Here, a tree-based approach is presented that associates to each path a probability of being executed (high-level SPTA), and a first technique that associates to every memory reference a probability of being cached (low-level SPTA) is introduced. Based on this latter technique, a first approach to the study of multi-paths programs is then presented. These two low-level SPTA techniques both were introduced in 2014, and thus constitute the state-of-the-art for this very new category of methods.

Determination of probabilities of tests outcomes using the input domain [10]

For this technique to be usable, one needs the AST of the structured program under study. Moreover, it is assumed that the WCET $W(B_i)$ of every basic block B_i is known, and that every loop has a number of iterations bounded by a known integer k. Finally, the input domain of the program must be known, and the probabilities of the possible values in this domain must be provided. This technique returns for every path an execution time, and a probability of being executed.

The idea with this technique is to determine for every test in the program the probability that it is true or not, considering the probabilities of occurrences of the possible inputs of the program. The probabilities are then combined to derive the probability for each path to be executed. This is done as follows:

1. A safe execution time is associated to every basic block.
2. Since a path is determined by the tests that are taken, the probability for every indirection to be taken is computed by considering the possible input domain of the program under study.
3. A tree-based approach similar to the one presented in Section 2.1.1.1 is used to determine the execution time profile of the program.

This method uses probabilities to take into account the uncertainty that is related to the inputs of the program. This method enforces the consideration of all possible paths by associating a probability of being taken to every test in the program. Therefore, a worst-case execution path is included, and has a probability of being executed. For this reason, safety is ensured provided that the probabilities associated to the input domain are correctly known.

SPTA for single-path programs and random replacement caches [2]

Since this technique falls into the category of *low-level* SPTA methods, the details of the caches must be known. In [2], the authors assume a N-way associative cache with random *evict on miss* replacement (a random line is evicted from the cache only if the required information is not inside). Moreover, this method applies only to *single-path* programs.

Given a trace of memory references in a single-path program, the authors of [2] first review the various equations that appear in the litterature to determine the hit probability of a memory reference. They show that those proposed by Zhou in [28] (used in [1]) and Kosmidis *et. al* in [15] can lead to an optimistic probability of hit for a given reference or sequence of references (*i.e*, the methods can return a probability of hit that is higher than the actual one).

As an answer to these overestimating equations, the authors define two criteria that must be met for an execution time profile not to overestimate the real probabilities of occurrence of execution

times, and thus not to overestimate the probability of exceedance for a given execution time:

- *Local safety*: The estimated hit probability of a given reference must be lower than the real one.
- *Independence*: The convolution of the estimated hit probabilities must produce an execution time profile that does not overestimate the real execution time profile (*global safety*).

To achieve these two objectives, they propose to use a locally safe estimator, and to *cut* the probability to the cache associativity (*i.e* they assume that the hit probability for a reference not being reused for a certain number of references is null). Thus, they propose the following estimator $\hat{P}^{\mathtt{M}}(k)$ to describe the probability of a hit in a cache, with k being the reuse distance of the memory reference `M` (*i.e* the number of dynamic memory blocks referenced since the last occurrence of `M`) and N being the associativity of the cache:

$$\hat{P}^{\mathtt{M}}(k) = \begin{cases} \left(\frac{N-1}{N}\right)^k & \textbf{if } N > k \\ 0 & \textbf{otherwise} \end{cases}$$

In this equation, $\left(\frac{N-1}{N}\right)^k$ represents a lower bound on the probability for a memory reference to be in the cache after k intermediate memory accesses ($\frac{N-1}{N}$ is the probability to remain in a cache with random replacement after a memory reference causing a miss). The second part limits the reuse distance to the associativity of the cache, but introduces pessimism since it lowers the probability of a hit in the cache to 0 after k intermediate memory references.

The individual hit probabilities are then combined using a convolution operator, as it is done in Section 2.2.2 to provide joint probabilities representing the probabilities for sequences of references to occur.

The authors have shown in [2] that the pessimism introduced by the cut to zero is essential not to overestimate the probability of hits in a sequence of references. However, they demonstrate only on an example in which the cut is essential, and do not provide a full proof that it prevents optimism in the general case. They also introduce in [2] a more precise equation based on so-called *cache contention*. Since it is less intuitive than the *cut* at cache size and it is not proven in the general case, it will not be considered here.

After defining this equation, the authors introduce an algorithm to mix an exhaustive enumeration of cache states with the application of the equation, in order to limit the precision loss. The idea here is to use the precise exhaustive method for well-chosen memory blocks (a set R including the m memory references with the highest occurrences count in the trace), and to keep the imprecise equation for the others. This combined approach allows to limit the precision loss caused by the use of the equation, while limiting the exponential complexity of enumerating cache state (thanks to the limit to k intermediate references in the equation).

This approach has the advantage to use probabilistic aspects only to describe the behavior of randomized hardware, and provides probabilities for every memory reference to be cached. It can then be used together with a high-level method to provide a WCET estimate that takes into consideration the probabilistic aspect of the underlying platform.

This method however is still in its development process. It considers only single-path programs, which is a very limited class of programs. Moreover, the safety of the proposed equation is not fully proven in the general case.

SPTA for multi-path programs and random replacement caches [11]

The previous method is the first method to provide a pWCET with probabilities that takes into account the random aspects of the underlying hardware. However, as stated before, it only handles *single-path* programs, which is a very small class of programs. In [11], the authors propose a sketch of a method[8] to provide a pWCET for programs with more than one possible path. They assume that the CFG of the program is available, that it is correctly structured, and that every loop has a bounded number of iterations.

The idea here is to *collapse* the structures in the CFG so that they can be represented by a single synthetic path. In other words, the authors replace regions of the CFG that have multiple paths with a single path whose execution time profile is a safe approximation of the one of the replaced region. Figure 11 (taken from [11]) presents the application of the proposed method to an example CFG composed of a `if` structure inside a loop. In this example, the various $\mathbb{Q}$ represent the synthetic versions of the paths that appear in the program.

Figure 11: Example of application of the *collapsing* process. A synthetic path $\mathbb{Q}^L$ is computed by first collapsing the two branches of the "if" structure, and then by iterating the maximum number of times on the loop.

As it was already stated, this approach is a first attempt to apply SPTA to multi-path programs, while keeping the probabilities to take into account the random aspects of the hardware. However, the authors of [11] use the method in [2] as a reference to compute the execution time profiles of the synthetic traces. The safety of this method is thus directly linked to the validity of the estimator proposed in [2].

[8]The complete work is currently under review, but is not published yet.

2.3 Motivations for our work

In this section, several methods to derive the WCET of real-time programs were presented. They have been classified into two main categories: deterministic and probabilistic techniques.

Deterministic techniques provide a single WCET estimate for a given program. They can be separated into static and measurement-based techniques. The latter use measurements to compute the WCET estimate, while the former rely on models of the program and/or of the architecture to derive a WCET estimate without executing the program.

Probabilistic techniques include all methods in which probabilities of occurrence are associated to timing values. They compute a pET distribution from which a CCDF can be derived to compute a pWCET (*i.e*, a bound on the execution time of the program that can be exceeded with a given probability). The probabilities capture the random aspects of the analyses, such as the distribution of input data, or the probability for a memory access to be cached when using a cache with random replacement. Table 1 summarizes all the studied methods according to the classification that was used, and puts in evidence (in red) that a safe SPTA method that uses the probability to model the impact of the random hardware is missing.

Technique	**Det.**	**Proba.**	**Static**	**Dynamic**	**Safe**	**Comments**
Tree-based computation [22]	x		x		x	High-level
IPET [18]	x		x		x	High-level
Analysis using abstract interpretation [13]	x		x		x	Low-level
Automatic generation of tests for exhaustive paths coverage [27]	x			x	x	
Heuristics and genetic algorithms [26]	x			x		
EVT-based approach [9]		x		x		Safety depends on paths coverage
Paths execution probabilities computation [4]		x		x		Uncertainty from hardware Safety depends on paths coverage
Determination of probabilities of tests outcomes using the input domain [10]		x	x		x	Uncertainty from input data
SPTA for single-path programs and random replacement caches [2]		x	x		?	Uncertainty from hardware, single-path
SPTA for multi-path programs and random replacement caches [11]		x	x		?	Uncertainty from hardware, multi-path
SPTA for multi-path programs and random replacement caches		x	x		x	Uncertainty from hardware, multi-path

Table 1: Summary of the existing methods for WCET estimation of real-time programs.

As it can be seen, no fully proven probabilistic static method for timing estimation of multi-path

programs currently exists that takes into account the randomized aspects of the hardware. Such a method would allow the determination of a pWCET estimate for the program under study that would be more trustworthy than the one provided by MBPTA techniques because the probabilistic aspect would come from the randomized architecture, and not from paths coverage.

This document aims at filling this need, by providing a way to compute a pWCET for single-path programs, using methods that guarantee its safety.

3 Exhaustive cache states enumeration per windows

In this section, we present an approach that is different from the one introduced in [2] to compute a pWCET for a single-path program. In Section 2.2.3, we have seen that current state-of-the-art SPTA methods perform in two steps. First, a locally safe estimation of the hit probability is found for every memory reference. Then, these references are combined using an operator such as convolution to determine a safe estimation of the probabilities of sequence of references, thus providing a pWCET for the whole program.

We have also seen in Section 2.2.3 that this approach is not proven to ensure safety. To avoid being dependent on the assumption that a cutting method ensures safety, we do not work with a two-step approach, and perform an analysis that enforces safety by construction.

The idea is as follows: given a single-path program, we split the trace associated to it into multiple small sub-traces (using a heuristic method). Then, an exhaustive cache states enumeration is performed for each of them to obtain safe estimates of the pET distributions associated to these sub-traces. These analyses are performed in a way that enforces independence (we use the same initial cache state for every analysis), thus convolution can be applied between these individual results to provide a safe pET distribution for the whole program.

This section is organized as follows: Section 3.2 introduces the method that we propose, and presents its main features. Section 3.3 studies the choice of the initial cache state to use for analyzing every window in order to determine the initial state leading to the worst-case pET distribution (*i.e* the pET distribution associated to a CCDF that is higher than for every other initial cache state). Section 3.4 discusses the termination, complexity and safety of the method, and Section 3.5 presents the heuristics that we propose to be used by our technique.

3.1 Assumptions and vocabulary

The proposed method works for single-path programs, from which a finite trace of memory references can be extracted. The CFG associated to the program is only needed by some of the heuristics presented in Section 3.5. We also assume that every loop in the program iterates the maximum number of times (since we are interested in the WCET, this assumption is quite natural).

We work on a N-ways set-associative cache with modulo placement and random evict-on-miss replacement policy. Such caches work as follows: every memory reference has an associated set that allows to determine in which portion of the cache the address will be placed. When an address is referenced, if it is not a hit, it evicts one block from the same set at random to take its place. A N-ways set-associative cache has N blocks per set (addresses are loaded to the cache by blocks of consecutive addresses in the memory, the first address of every block is called its base address).

In this document, we focus on the impact of instruction caches on the WCET only. Although, the method may be applied for data cache but optimized heuristics would have to be defined. Others aspects such as pipelines is out of the scope of this work.

In the remaining of this document, we will use the term *trace* to refer to the sequence of addresses

associated to the full program. The addresses in a trace hitting to a same set s will be called a *sub-trace*. Finally, elements of the partition of a sub-trace using one heuristic from Section 3.5 will be called *windows*. Moreover, the traces studied here will only be composed of the base addresses of the references, since it is equivalent.

3.2 Presentation of the method

The main objective of our method is to ensure global safety, while being as close as possible to the real probabilities of occurrence for the possible execution times. A very simple way to achieve such an objective would be to perform an exhaustive cache states enumeration following the trace:

1. Assume an initial cache full of references that do not appear in the considered program (this choice aims at providing a distribution for the worst-case initial state, and is discussed in Section 3.3).
2. For every memory reference, determine the cache states that result from the execution of this reference, and their associated probabilities of occurrence[9]. This will result in a tree having at the leaves the possible cache states after the whole program is executed.
3. For every possible path from the root of the tree to its leaves, determine its probability and associated execution time ($nbMisses * latencyMiss + nbHits * latencyHit$). These times form a pET distribution.

It is quite evident that such a method is untractable for long traces due to complexity reasons. As an example, Figure 12 shows the tree associated to the simple trace `A;B;C;A;C;B` (assuming a set-associative cache of 4-block lines, and that `A`, `B` and `C` hit in the same set).

In this example, the execution in which three references cause a miss and three cause a hit is only represented by the leftmost branch. Therefore, if the miss latency is of 10 cycles and the hit latency is of 1 cycle, the probability for the program to last $3 * 10 + 3 * 1 = 33$ cycles is equal to $1 * \frac{3}{4} * \frac{1}{2} * 1 * 1 = \frac{3}{8}$. The whole pET distribution associated to this tree is shown in Figure 13.

By analyzing the tree in Figure 12, we can notice that the augmentation of the number of cache states comes from the misses in the trace. This implies that this exhaustive analysis might be tractable on well-chosen portions of a trace in which there is a lot of reusability, since the hits in the cache do not impact the width of the tree.

This observation constitutes the main idea behind our approach. For a single-path program to study, we want to split the associated trace into a partition of smaller windows[10] on which we independently perform the previously presented exhaustive analysis (all analyses start from the same cache state, in which none of the references from the window is present). These analyses thus compute for every window pET distributions that are pairwise independent. Finally, convolution

[9] Our implementation merges leaves that represent the same cache contents and same associated timing values to minimize the number of leaves.

[10] The windows must fully cover the trace without overlapping, and must keep the order in which the references are executed in the trace for every set.

Figure 12: Exhaustive cache states enumeration for the trace of references `A;B;C;A;C;B` hitting in the same set s (which possible contents are represented by the four-cell arrays). The references resulting in misses are depicted in red and the ones resulting in hits are in green. The probability of occurrence for every cache state is obtained by multiplying the probabilities on the edges leading to it from the root.

Figure 13: pET distribution associated to the the trace of references `A;B;C;A;C;B` for which the tree is depicted in Figure 12, assuming a hit latency of 1 and a miss latency of 10. The probabilities of occurrence of the various execution times are obtained by summing the probabilities of the associated branches to occur.

can be applied between these pET distributions to provide the pET distribution associated to the whole trace.

It is also interesting to notice that in the case of a N-way set associative cache, references hitting in different sets never interact (*i.e* a reference `A` associated to set 0 that causes a miss has no possibility to evict a reference `B` from the cache if `B` is associated to set 1). We can use this observation at our advantage to virtually reduce the size of the trace under study. Since addresses associated to different sets do not interact, we split the trace under study into some sub-traces corresponding to the references in the main trace associated to a given set. The exhaustive analysis can then be performed independently on these sub-traces, and the associated pET distributions can then be convoluted to obtain the pET distribution associated to the single-path program without any loss in precision due to this separation into sub-traces.

Algorithm 1 sums up the ideas previously described, and along with the heuristics proposed in Section 3.5 constitutes the main contribution of this work. In this algorithm, N represents the associativity of the cache. Some methods in it are not detailed for the sake of comprehension. Ideas of possible implementations for these functions are given when studying the termination and complexity of the program.

We can notice that the precision of the method is directly related to the heuristic used. Splitting a sub-trace into windows implies that the cache contents is reinitialized for each of them. Therefore, every first occurrence of a reference in a window will be considered a miss for sure, which introduces pessimism. We can thus state that the bigger the windows, the higher the precision and also the complexity. Therefore, the heuristic used for splitting has to make a trade-off between precision and computational time.

As an example, if we study a trace `A;B;C;B;C` and the heuristic returns the windows `[A;B;C][B;C]`, the reference to `B` in the second window will result in a miss with probability 1 although it might

Algorithm 1: $exhaustiveAnalysisOfWindows\ (trace)$

```
// We remember all the pET distributions
dists := {}
foreach n ∈ [1; N] do
    // We get the sub-trace of references associated to set n
    subTrace := extractSubTrace(n, trace)
    // We use one heuristic among those presented in Section 3.5 to split the
    sub-trace into windows
    windows := determineWindows(subTrace)
    // We determine the pET distributions using the method illustrated in
    Figure 12
    foreach w ∈ windows do
        initialCache := initCacheWithNoReferenceFromWindow(w)
        dist := exhaustiveEnumerationOfCacheStates(w, initialCache)
        dists := dists ∪ {dist}
// We return the convolution of the pET distributions
return ⨁ dists
```

still be cached (if `C` evicted `A` for example).

3.3 Initial cache contents

We previously stated that the pET distribution determined by the exhaustive cache states enumeration should be computed with an initial state leading to the worst-case pET distribution for every window (*i.e*, the pET distribution associated to a CCDF that cannot be higher for another initial cache state). We can group the possible cache states into three categories:

1. The cache is not full. In this case, a reference `A` not in the cache and associated to a set n will not evict a reference `B` already present in n, and will instead be loaded in an empty cache line[11].

2. The cache contains a reference `A` appearing in the trace. In this case, there exist some paths in the exhaustive tree in which the reference to `A` will lead to a hit eventhough it is encountered for the first time.

3. The cache is full, but contains only references that do not appear in the trace.

As an example for the first case, if we assume a sequence `A;B;A` with `A` and `B` associated to an empty set n of at least two lines, then the second occurrence of `A` will be a hit with probability 1, since `B` would be placed to an empty cache line in n. However, in the case in which the cache is full of references not including `A` and `B`, the second `A` might be a miss. Therefore, this first category of caches may cause overestimations of hit probabilities.

[11] This behavior depends on the *fill policy* of the cache. There also exist cache implementation that consider empty lines as if they were full when evicting something.

It is quite obvious that the second category does not lead to the worst-case pET distribution, since it may for example already contain the first reference of the trace. In this case, we have for sure that this reference will be a hit. However, with the third category of cache, this reference would have led to a miss. Therefore, the second category does not lead to the worst-case pET distribution.

In the last case, the initial probability of hit for every reference is 0, which is clearly the most pessimistic estimation for the probability to hit. Thus, the hit probabilities for the various references in the trace only depend on the previously executed references, and not on the initial cache contents. Therefore, any cache state from this third category will lead to the worst-case pET distribution.

As a conclusion, any cache being full of references that do not appear in the trace under study (or equivalently not appearing in the window under study) can be used as an initial cache for the exhaustive enumeration of cache states, and will lead to the worst-case pET distribution. Other cache states lead to an overestimation of the hit probability for some references, and thus trade-off the safety of the generated pET distribution.

3.4 Termination, complexity and safety considerations

Here, we study Algorithm 1 in more details to prove its termination. Additionally, we examinate it to determine its complexity. Finally, we provide evidence of the correctness of the method. The tightness of the method will be examinated experimentally in Section 4.

3.4.1 Termination of the method

Since the cache has a fixed number of sets, iterating over all sets will eventually terminate. For each iteration, we must prove that the various methods used also terminate:

- $extractSubTrace$: Extracting the sub-trace of references associated to a given set n can be done by iterating over the whole trace, and keeping only the references associated to n. Since the trace is finite, the method terminates.
- $determineWindows$: This method is a heuristic, which will be detailed in Section 3.5. Termination will also be studied there. Anyway, this heuristic method splits a finite sub-trace into a partition, and therefore the loop iterating over the winwows will eventually terminate.
- $initCacheWithNoReferenceFromWindow$: Initializing the cache with references not being present in the window can be done by iterating over the references in the window and remembering which ones should not be put in the original cache state. Since the windows are finite, the method terminates.
- $exhaustiveEnumerationOfCacheStates$: Given an initial cache state, the method iterates over the references in the window and computes at each iteration a finite number of cache states. Since the windows are finite, the method terminates. Moreover, the method returns a pET distribution that contains a finite number of associations $(time, probability)$, since there is a finite number of paths in the associated tree.

The convolution operator $\oplus$ is the same as presented in Section 2.2.2. It consists of a finite number of products and sums. Therefore, since the pET distributions returned by the exhaustive enumeration of cache states are finite (and there is a finite number of them, one per window), performing their convolution terminates.

As a conclusion, provided that the heuristics introduced in Section 3.5 terminate, Algorithm 1 terminates.

3.4.2 Complexity of the method

The sub-traces issued from $extractSubTrace$ form a partition of the whole trace. Moreover, the windows issued from a given sub-trace also form a partition of this sub-trace. Therefore, the two loops in Algorithm 1 iterate only once over each reference of the original trace. The complexity therefore depends on the methods in the body of the loops.

As stated in Section 3.4.1, the methods $extractSubTrace$ and $initCacheWithNoReferenceFromWindow$ can be done by iterating respectively once over the trace, and once over the window under study. Therefore, each of these methods perform a number of operations that is linear in the number of references.

The complexity of the heuristic method $determineWindows$ will be studied in Section 3.5.

Finally, the complexity of $exhaustiveEnumerationOfCacheStates$ is exponential in the number of potential misses (*i.e* references having a hit probability that is not 1), since a reference causing a miss in the cache will cause an eviction and will lead to N new possible cache states. However, for a given reference `A` to execute, not all possible cache states will result in a miss (since `A` might have been already executed before). Therefore, the complexity is bounded by an exponential in the number of misses if there is some reusability, and this bound is reached for sequences containing only distinct references.

As a conclusion, the complexity of Algorithm 1 is equal to the maximum between:

- The complexity of the heuristic method chosen for $determineWindows$ to split a sub-trace into windows (multiplied by the associativity of the cache).
- The complexity of the exhaustive method $exhaustiveEnumerationOfCacheStates$ that is at most $O(N^{|T|})$, where $|T|$ is the number of references in the trace under study (the bound is reached if all references are associated to a single set, if they all map to different cache lines, and if there is only one window containing the whole trace).

It is important to notice that the complexity here really depends on the size of the windows. For this reason, this complexity analysis aims at providing an idea of the individual complexities for the methods used, and experimental evaluation in Section 4 is more representative of the complexity for the whole program.

3.4.3 Safety of the method

As it was stated several times in this document, safety is the most important property to ensure. We say that a pET distribution is safe if it respects the definition in Section 2.2.1.3. This is the case if the pET distribution respects one of the following criteria:

- It is built from the leaves of an exhaustive tree in which all hit probabilities do not overestimate the real hit probabilities.
- It is generated by the convolution of independent safe pET distributions.

The method performs an exhaustive analysis of the possible cache states starting with the worst initial cache state (as defined in Section 3.3), so the execution leading to the worst-case execution time is captured by construction if the whole program is processed in one tree (as in Figure 12). By splitting the program into windows and resetting the cache for each of them, we force the probability of every reference in the initial state to be 0, which is clearly an under-approximation of the real probability to be cached. Therefore, only potential hits will be considered misses but not the opposite, and the generated pET distribution will be safe.

Since we reset the cache contents for the analysis of every window, it is equivalent to performing as many independent analyses as there are windows (as if we were studying multiple distinct programs). Therefore, the pET distributions that are obtained are completely independent, and the requirements to perform convolution are fulfilled.

As a conclusion, the method provides a safe pET distribution for every window (the loss in precision comes from the initialization of the cache contents. Without this precision loss, the pET distribution provided by the method is exact). Then these estimates are combined using convolution[12], which ensures safety for the pET distribution associated to the whole program since the individual pET distributions are independent.

3.5 Various heuristics for the method

As it can be seen in Algorithm 1, a heuristic method $determineWindows$ needs to be defined in order to cut the various sub-traces into a partition of smaller windows. This step is essential to bound the size of the windows to reduce the exponential complexity of the analysis, and make the exhaustive analysis of cache contents tractable.

In this section, we present three heuristics that can be used as an implementation for $determineWindows$. For each of them, we present how it works, and study briefly its termination and complexity.

[12]Resampling methods as introduced in Section 2.2.2 can also be used as long as they keep the safety property, as presented in [20].

3.5.1 Fixed number of dynamic references per window

3.5.1.1 Presentation of the heuristic

The first heuristic is the simplest one. We have seen in Section 3.2 that the width of the tree is related to the number of misses that occur along the trace. Therefore, limiting this parameter is essential to make the exhaustive analysis tractable. A simple way to do this is to iterate over the trace and to create a window everytime W potential misses (*i.e* references with a probability of hit lower than 1) are found.

This heuristics performs as presented in Algorithm 2. In this algorithm, the parameter W corresponds to the maximum number of misses allowed per window. The idea is to iterate over the sub-trace and to create a window every time that W dynamic memory blocks are encountered. The impact of the value of W on the precision and execution time is studied in Section 4.2.

Algorithm 2: *determineWindows* (*subTrace*)

```
// Initialization of the variables (windows will contain the final result)
windows := {}
currentWindow := {}
nbMisses := 0
previousRef := NULL
// We iterate over the sub-trace and create a new window every W misses
foreach ref ∈ subTrace do
    // We change the window if the current window contains W misses
    // We still fill the window if ref is a hit for sure
    if nbMisses = W and baseAddress(previousRef) ≠ baseAddress(ref) then
        windows := windows ∪ {currentWindow}
        currentWindow := {}
        nbMisses := 0
        previousRef := NULL
    // We count the distinct references
    if nbMisses = 0 or baseAddress(previousRef) ≠ baseAddress(ref) then
        nbMisses := nbMisses + 1
    // We continue to fill the window
    previousRef := ref
    currentWindow := currentWindow ∪ {ref}
// We save the last window
windows := windows ∪ {currentWindow}
return windows
```

As an example, consider the sub-trace `A;B;C;A;A;B;A;C;B;C;A;D;A;B;A;A` with $W = 4$. The algorithm will iterate over the sequence and will generate the partition `[A;B;C;A;A][B;A;C;B][C;A;D;A][B;A;A]`.

3.5.1.2 Termination of the heuristic

Considering the termination of this heuristic, it consists of a simple loop that iterates over the sub-trace. Therefore, since the trace is of finite size, the algorithm terminates.

3.5.1.3 Complexity of the heuristic

All the computations that are performed in the loop (*baseAddress*, $\cup$) can be made in constant time. Therefore, Algorithm 2 has a linear complexity (in the number of references in the sub-trace).

3.5.2 Windows around the loops

Additionally to the trace corresponding to the program, the control flow graph (CFG) associated to it must be available to be able to perform this heuristic. It can either be determined at compilation time, by analyzing the associated source file, or sometimes retrieved from the trace.

3.5.2.1 Presentation of the heuristic

The heuristic introduced in Section 3.5.1 has the disadvantage to be completely uncorrelated to the structure of the program. It just processes sequentially along the sub-trace and determines some windows by counting the number of possible misses.

Consider the sub-trace `A;B;C;D;E;F;E;F;E;F` which is typical of a sequential code `A;B;C;D` followed by a loop `E;F` iterating three times. If $W = 6$, the first heuristic would have generated the windows `[A;B;C;D;E;F][E;F;E;F]`. With this partition, the first iteration of the loop is a miss (since `E` and `F` were not referenced previously), and the second iteration is also a miss (because the cache is reinitialized for the analysis of every window).

This example illustrates the fact that the reuse of memory references comes from some structures that can be determined using the CFG: the loops and the function calls. The loops are the most natural way to repeat a reference multiple times. The function calls also cause the same references to be used multiple times if a function is called more than once.

This second heuristic focuses on the first category of structures: the loops (integrating the function calls will be done in Section 3.5.3). The idea is to place the bounds of the windows so that they fully encapsulate the iterations of the loops, starting from the outermost loops (those encapsulated by no other loops). Then, for every window that is defined, if it is too big[13], it is splitted into windows associated to the inner loops, and so on until the windows are small enough, or until there are no more inner loops to consider. In this last case, the windows may be too big, and a call to the first heuristic introduced in Section 3.5.1 can be used to reduce its size.

[13] Determining if a window is too big can be done using the heuristic introduced in Section 3.5.1. If it returns only one window, then it is small enough.

Algorithm 3 details the various steps of this second heuristic[14]. In this algorithm, *heuristic*1 is a call to the heuristic defined in Section 3.5.1. It allows to limit the size of the windows to W potential misses. Additionally, *encapsulatesALoop* indicates if there exists a loop l having all its iterations being encapsulated in w. Finally, *placeWindowBoundsAroundInnerLoops* returns a partition of w such that every instance of l (not iteration) is placed in a new window (*i.e*, if a full execution of a directly nested loop l can be find in w, then it is placed in a window).

One can remark that if there is a loop l_1 encapsulating another loop l_2 like in Figure 14, then for a W that is too small, every iteration of l_1 will be put into a distinct window and reuse between iterations of l_1 will not be captured. In this example, with $W = 16$, the algorithm would process as follows:

1. Process the whole trace `A;B;C;D;C;D;C;D;E;B;C;D;C;D;C;D;E;B;C;D;C;D;C;D;E;F`.
   It is too big (26 misses). Put outermost loops into windows:
   `[A][B;C;D;C;D;C;D;E;B;C;D;C;D;C;D;E;B;C;D;C;D;C;D;E][F]`.

2. Process the window `A`.
   It is small enough. Continue to the next window.

3. Process the window `B;C;D;C;D;C;D;E;B;C;D;C;D;C;D;E;B;C;D;C;D;C;D;E`.
   It is too big (24 misses). Put loops directly encapsulated into l_1 into windows:
   `[A][B][C;D;C;D;C;D][E;B][C;D;C;D;C;D][E;B] [C;D;C;D;C;D][E][F]`.

4. All windows are small enough.
   This partition is kept.

Figure 14: Example CFG corresponding to a single-path program. It consists of two loops l_1 (iterating three times) and l_2 (iterating three times per iteration of l_1). In this program, l_1 is the outermost loop and l_2 is nested into l_1. Thus, if a window is placed around l_1, it will also include all the instances (and iterations per instance) of l_2.

However, one can notice that it is possible to merge the 5 first windows into one bigger window of size sufficiently small to be accepted (with less than W misses). This would allow to group

[14]Here again, our implementation performs optimizations, such as not iterating again over the sufficiently small windows.

multiple iterations of l_1 into a single window and therefore to improve the tightness of the analysis (since the cache would not be reinitialized at every iteration). For this reason, we use a function $mergeSmallWindows$ in Algorithm 3.

Moreover, since the references are written sequentially into a program, then it happens frequently that references at the end of the assembly code of a loop have the same base address as the references in the sequence immediately following them. Therefore, these references should be in the same windows not to miss some sure hits. To do this, we add a function $correctAddresses$ to Algorithm 3 that processes as follows: for two consecutive windows w_1 and w_2, move the references at the beginning of w_2 to the end of w_1 if they have the same base address as the last reference of w_1.

Algorithm 3: $determineWindows$ $(subTrace)$

```
// We initialize the possible windows with the whole sub-trace
windows := {subTrace}
// We reduce the windows until they are small enough or cannot change
repeat
    windowsBefore := windows
    foreach w ∈ windows do
        // We split a window if it is too big and includes iterations of a loop
        heuristic1Partition := heuristic1(w)
        if size(heuristic1Partition) > 1 and encapsulatesALoop(w) then
            // We place bounds around the loops that are directly nested in w
            newWindows := placeWindowBoundsAroundInnerLoops(w)
            mergedNewWindows := mergeSmallWindows(newWindows)
            windows := windows \ {w}
            windows := windows ∪ mergedNewWindows
until windowsBefore = windows;
// We correct the windows not to lose sure hits
correctedWindows := correctAddresses(windows)
// We use the heuristic in Section 3.5.1 to bound the size of the windows
finalWindows := {}
foreach w ∈ correctedWindows do
    finalWindows := finalWindows ∪ heuristic1(w)
return finalWindows
```

3.5.2.2 Termination of the heuristic

Algorithm 3 mainly consists of a `repeat until` loop stopping when there is no change between two of its iterations. In this loop, the `foreach` loop iterates over all the defined windows to reduce their size if they are too big (and if they can be reduced). Therefore, for every iteration of the `repeat until` loop, the size of every window decreases or does not change. Since the windows are of finite size, the `repeat until` loop will eventually terminate. Additionally, the `foreach` loop uses calls to the following methods:

- $heuristic1$: The termination of this method was previously studied in Section 3.5.1.2.

- *encapsulatesALoop*: This method can be implemented by checking in the CFG if a loop is included in the body of the loop represented by the window under study (or is an outermost loop if the window under study represents the whole sub-trace). Since there is a finite number of loops in the CFG, this method terminates.

- *placeWindowBoundsAroundInnerLoops*: As it is the case for method *encapsulatesALoop*, identifying the directly nested loops can be done by simply studying the loops in the CFG. Here, a second step consists of mapping those loops on the window under study. This can be done by iterating on the references in the window, and to put them into new windows if they belong to one of the identified loops. Since there is a finite number of loops, and since the window under study is of finite size, this method terminates.

- *mergeSmallWindows*: This method can be implemented by iterating on the new windows and by moving the first references of a window to the end of the previous one if they have the same base address. Since there is a finite number of windows and since they are of finite size, this method terminates.

The second `foreach` loop iterates over the new windows to limit their size using the heuristic introduced in Section 3.5.1. Since there is a finite number of new windows (partition of the studied window) and since *heuristic*1 terminates, this second heuristic terminates.

3.5.2.3 Complexity of the heuristic

The `repeat until` loop stops when the windows are small enough, which is determined by checking if the number of windows that would have been generated if using the heuristic presented in Section 3.5.1 is equal to 1. Thus, the complexity of the heuristic is dependent on the value of the parameter W. Here, we want to study the complexity of the heuristic, independently from the analysis that performs it. Therefore, we fix $W = 1$ to consider the complexity in the worst case.

Since every window must be cut until it is small enough, it can be cut at most n times, where n is the number of references in it. Therefore, the whole sub-trace can be cut once per reference. Thus, the contents of the inner `for` loop can be executed as many times as there are references in the trace. The complexity of its contents is as follows:

- *heuristic*1: It was previously shown in Section 3.5.1.3 that this method is linear in the number of references in the sub-trace.

- *encapsulatesALoop*: Checking if a loop nests another loop can be done by checking for every loop if all its nodes (at the CFG level) are included in the nodes of the first one. This is thus linear in the number of loops, which is much lower than the number of references.

- *placeWindowBoundsAroundInnerLoops*: Mapping the loops on the window can be done by determining the loops (using the same technique as *encapsulatesALoop*), and then mapping them on the window under study. This second step can be done by iterating on the references of the window to classify them. Thus, this method is linear in the number of references.

- *mergeSmallWindows*: This method can be implemented by iterating on all windows once. Thus, this method is of linear complexity in the number of references.

As a conclusion, this heuristic iterates at most a number of times equal to the number of references in the sub-trace. For each of its iterations, methods of linear complexity are called. Thus, the complexity if this second heuristic is quadratic in the number of references of the sub-trace.

3.5.3 Windows for the independent parts of the trace

The idea of this third heuristic is to capture the portions of the sub-trace in which some references are reused. As a matter of fact, separating a sub-trace into windows having no reference in common does not impact the precision of the analysis. Then, for windows that are too big, the second heuristic presented in Section 3.5.2 can be used to bound their size.

This heuristic uses the second heuristic presented in Section 3.5.2. Therefore, the CFG of the program under study must be available. If it is not the case, then the call to *heuristic*2 in Algorithm 4 can be replaced by a call to *heuristic*1 (corresponding to the first heuristic presented in Section 3.5.1). However, precision of the analysis will be reduced for the reasons explained in Section 3.5.2.

3.5.3.1 Presentation of the heuristic

As stated before, the reutilisation of the references comes from the loops and the function calls. The heuristic presented in Section 3.5.2 aims at capturing the reuse caused by the loops. This third approach aims at also taking in consideration the reuse of the references caused by function calls.

To do so, the idea is to find parts of the sub-trace that are independent of others. We consider that two windows are independent if they share no references having the same base address. Thus, if such windows can be extracted from the sub-trace, it implies that no precision loss will result from the separation in windows when performing the exhaustive cache states analysis. We have previously seen that the loss in precision comes from the fact that reinitializing the cache resets non-zero probabilities that would have been useful later. In this case, since the references from a window are never reused in another one, this reinitialization does not impact the analysis.

However, consider a sub-trace that starts and finishes by a call to the same function. All the references in the sub-trace are thus put in a unique window. Therefore, this third heuristic does not guarantee that the windows are of sizes sufficiently small for the exhaustive analysis to be tractable, and a call to a heuristic like the one introduced in Section 3.5.2 must be done (which will in turn use the heuristic introduced in Section 3.5.1).

The algorithm to separate the sub-trace into independent windows is quite simple. By iterating on the sub-trace, one can remember the encountered memory blocks, and check if they appear again later in the sub-trace. If none of them is found, then a window is created.

The details are given in Algorithm 4[15]. In this algorithm, the call to *heuristic*2 refers to the use of the heuristic introduced in Section 3.5.2. Moreover, $subTrace[a, b]$ will return the references in $subTrace$ from the a^{th} (not included) to the b^{th} (included).

Algorithm 4: *determineWindows* (*subTrace*)

```
// Initialization of the variables
windows := {}
currentWindow := {}
// We iterate over the sub-trace and create a new window when independent
parts are detected
foreach ref ∈ subTrace do
    // We add the reference to the current window
    currentWindow := currentWindow ∪ {ref}
    // We find the base addresses of the references already encountered
    previousReferences := {}
    foreach previousRef ∈ currentWindow do
        previousReferences := previousReferences ∪ {baseAddress(previousRef)}
    // We find the base addresses of the references still not encountered
    futureReferences := {}
    foreach futureRef ∈ subTrace[size(currentWindow), size(subTrace)] do
        futureReferences := futureReferences ∪ {baseAddress(futureRef)}
    // We change the window if no encountered reference will appear again
    if previousReferences ∩ futureReferences = ∅ then
        windows := windows ∪ {currentWindow}
        currentWindow := {}
// We use the heuristic in Section 3.5.2 to bound the size of the windows
finalWindows := {}
foreach w ∈ windows do
    finalWindows := finalWindows ∪ heuristic2(w)
return finalWindows
```

3.5.3.2 Termination of the heuristic

The first loop of the heuristic in Algorithm 4 iterates over the whole sub-trace, which is finite. Then, for every iteration, the encountered and remaining addresses are checked. Since the sub-trace is finite, iterating a finite number over its contents will terminate.

The size of the windows is then limited using the heuristic presented in Section 3.5.2. Its termination was previously shown in Section 3.5.2.2. Therefore, this third heuristic terminates.

[15] As it was the case for the other algorithms, the implementation features some optimizations, such as avoiding recomputing the encountered addresses in the first nested `foreach` loop.

3.5.3.3 Complexity of the heuristic

This third heuristic iterates over the trace to determine for every reference the window in which it should be placed. For every reference, it checks the previous and future references to see if the intersection of their base addresses is not null. Therefore, the complexity of the first part of this heuristic is quadratic in the number of references in the sub-trace.

This heuristic then uses a call to *heuristic*2, which was also shown to be quadratic in Section 3.5.2.3. As a conclusion, the complexity of the whole heuristic is quadratic in the number of references in the sub-trace under study.

4 Evaluation of the method

4.1 Experimental setup

For all the analyses, we assume a modulo placement, random evict-on-miss replacement, set-associative cache of 8 sets and 4 ways, with 32B lines (1KB cache). This small size was chosen to adapt to the size of the programs under study (standard caches are too big, and might contain the whole program).

In order to count the number of misses when performing the different analyses, we set the cache hit latency to 0 cycles, and the miss latency to 1 cycle. From this number of misses and using the total number of references in the program, one can then easily determine the total latency for any chosen values of hit/miss latency using the following formula:

$$totalLatency = nbMisses * missLatency + (nbReferences - nbMisses) * hitLatency$$

We present the experimental evaluation of our method on the following programs from the Mälardalen WCET benchmark suite. From these programs, we extract the longest trace from the CFG (by iterating the maximum number of times in the loops). Some of the programs in Table 2 are multi-path. In this case, we fix one trace and work on it as if the program was single-path.

Name	**Description**	**Single or multi-path**	**Number of references**	**Potential misses**	**Maximim number of distinct blocks per set**
bs	Binary search for an array of 15 integer elements	multi-path	200	12	2
edn	Finite Impulse Response (FIR) filter calculations	single-path	442134	25221	19
fdct	Fast Discrete Cosine Transform	single-path	6550	833	14
jfdctint	Discrete-cosine transformation on a 8x8 pixel block	single-path	7523	714	12
lcdnum	Read ten values, output half to LCD	multi-path	965	204	3
ludcmp	LU decomposition algorithm	multi-path	27037	2035	13
matmult	Matrix multiplication of two 20x20 matrices	single-path	424487	2549	4
minver	Inversion of floating point matrix	single-path	4242	200	18
nsichneu	Simulates an extended Petri Net	multi-path	22010	2753	173
sqrt	Root computation of quadratic equations	multi-path	1605	166	3

Table 2: Reference single-path programs from the Mälardalen WCET benchmark suite.

In Table 2, the number of potential misses indicates how many references may be a miss (which is computed by counting the number of references non-consecutively associated to the same block). This is what is limited by our approach with the parameter W to bound the exponential complexity. The last column indicates the maximum number of distinct memory blocks that are mapped to a set according to the placement policy. This last parameter corresponds to what is limited in the state-of-the-art approach [2] with the parameter m (number of most occurring references, presented in Section 2.2.3).

In this section, we first study in Section 4.2 the impact of the value of W (limiting the number of potential misses per window) in the heuristics presented in Section 3.5 on the precision and

computation time of the analysis. Then, we compare our results to the current state-of-the art probabilistic methods in Section 4.3 using the CCDFs. The analyses are performed on a computer with 16GB RAM and an Intel Core i7-2820QM CPU processor with 8 cores at 2.30GHz.

When using our algorithm, all computations will be performed using the third heuristic presented in Section 3.5.3, since it captures more reuse of memory references than the two other ones. Because of this, it is the most precise. To confirm this, Figure 15 shows the CCDFs corresponding to the use of our algorithm with the three heuristics on program *edn*, with $W = 200$[16]. Comparison of our three heuristics on the other programs of Table 2 behave the same, thus results are not presented here.

In this figure, the red curve corresponds to the CCDF (introduced in Section 2.2.1) obtained by simulating the program 10000 times. Simulation provides an approximation of the real distribution, and allows to put in evidence experimentally the safety of the method. The dramatic fall in this curve is due to the limited number of runs. Indeed, modeling the real distribution of probabilities would require to perform an infinite number of runs to capture all the possible executions. Since the number of runs is reduced, so is the precision of the estimation using simulation. The blue curves correspond to the three heuristics presented in Section 3.5.

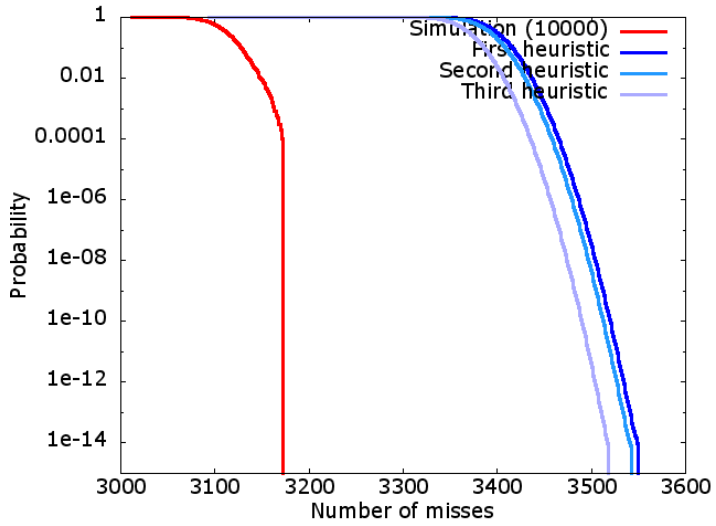


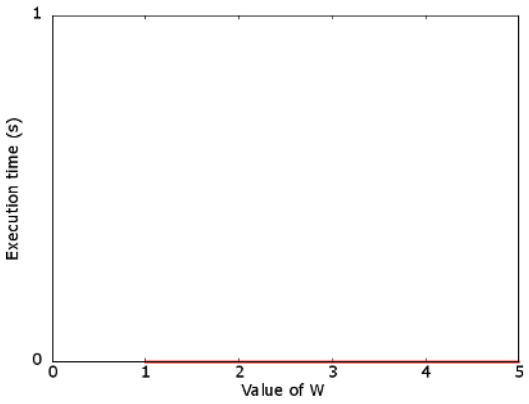
Figure 15: Comparison of the three heuristics presented in Section 3.5 in terms of precision on the program *edn* for $W = 200$.

4.2 Impact of the parameter W on the precision and computation time

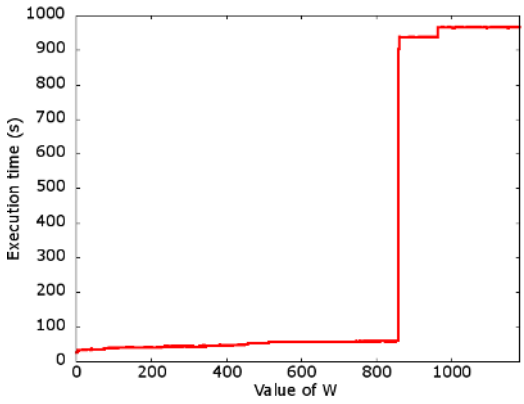
As we have seen in Section 3, the algorithm relies on a heuristic that uses a parameter W to bound the number of potential misses per window to exhaustively analyze. Since this parameter defines the complexity of the method, we experimentally study the impact of its value on the computation time and on the precision of the analysis.

[16]This particular value of 200 was chosen at random, since the behavior of the curves does not depend on W.

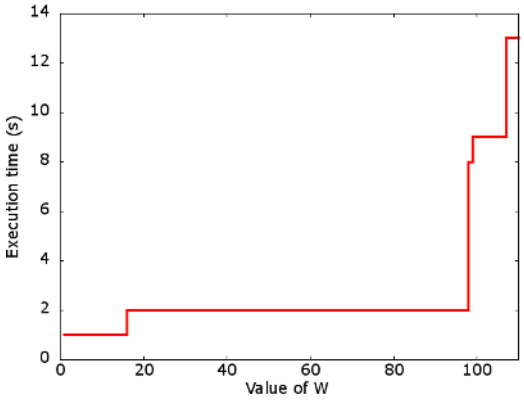
To evaluate the computation time, we consider our method with the third heuristic, and study the impact of the first values of W on the execution time of the analysis. Evaluating the impact of some of the values for W should be done to allow one to estimate a correct value for W to make a trade-off between precision and computation time (the resulting windows depend on the structure of the program, mostly because of the loops). Experimental results for the programs in Table 2 are given in Figure 17. In this figure, the x axis represents the various values for W, and the y axis shows the associated time needed for our algorithm to terminate.



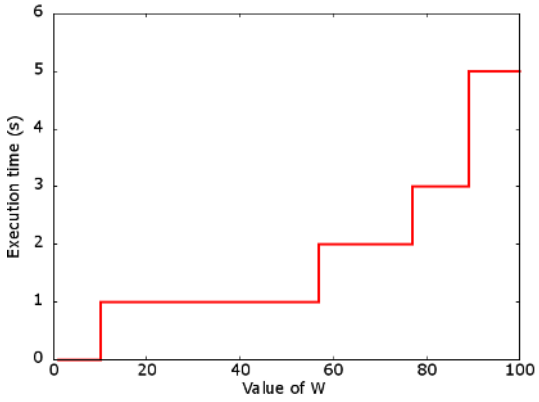
bs. Max W: 3 (reached)



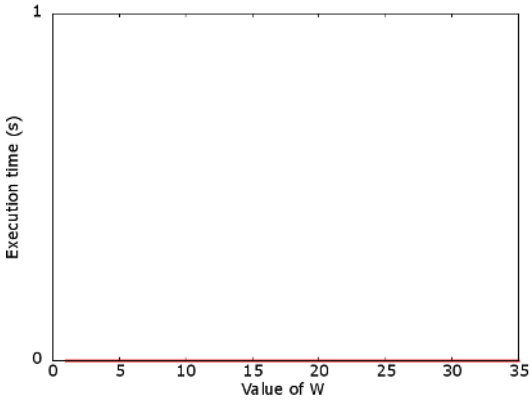
edn. Max W: 6401



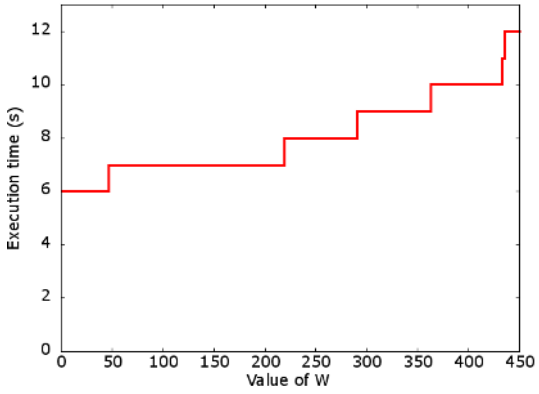
fdct. Max W: 107 (reached)



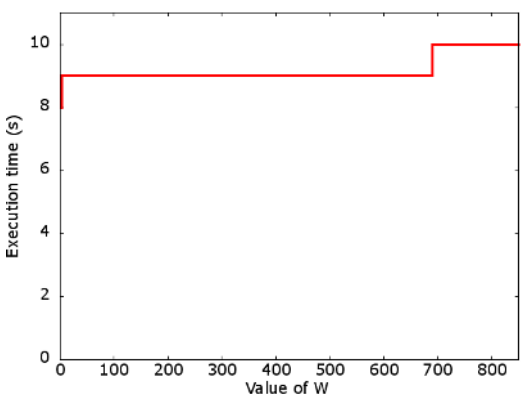
jfdctint. Max W: 90 (reached)



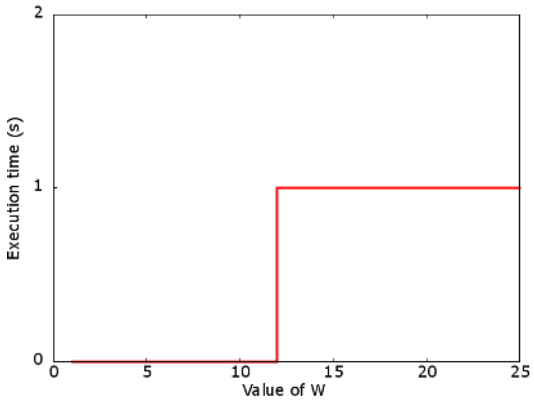
lcdnum. Max W: 31 (reached)



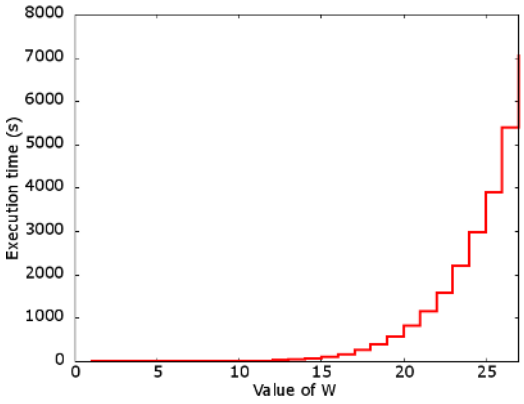
ludcmp. Max W: 434 (reached)



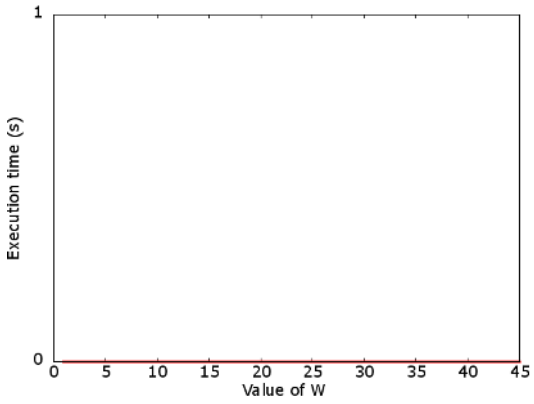
matmult. Max W: 844 (reached)



minver. Max W: 20 (reached)



nsichneu. Max W: 345



sqrt. Max W: 41 (reached)

Figure 17: Impact of W on the computation time for the analysis of the programs in Table 2.

We can see that the time exponentially increases with the value of W. This is of course because of the exponential complexity of the exhaustive analysis, as studied in Section 3.4.3. Moreover, we can see that the exponential grows most for purely sequential programs like *nsichneu*. Since there are no loops in the program, no reuse can be captured, and increasing W only results in allowing more misses to the analysis, thus increasing the width of the exhaustive cache states tree.

With the example of *edn*, we can see that there exist some steps in the curve. This is because the windows are the same for the distinct values of W. Such scenario happens in this case because there are multiple loops in the program: when W reaches a certain value, each loop execution is fully encapsulated in a window. Past this point, increasing slightly W does not allow merging these two windows in a bigger window. However, when W reaches a value that is big enough to merge the two windows, the computation time increases dramatically since the two small windows now form a unique big one.

We can also see that past a point, the computation time does not increase with W. This is the case for the benchmarks for which the maximum value of W is reached. This happens because once W is big enough, it causes the windows to contain the independent parts of the program. Therefore, increasing W does not change the windows.

Concerning the precision in function of the window size W, we compare in Figure 18 the CCDFs obtained from our approach (blue curves) for three chosen values of W for the benchmark *edn* (the other benchmarks behave the same). The chosen values for W correspond to three steps in the curve in Figure 17 In Figure 18, the x axis represents the number of misses as determined by the analyses. The y axis, in logarithmic scale, represents the probability to reach or exceed this number of misses.

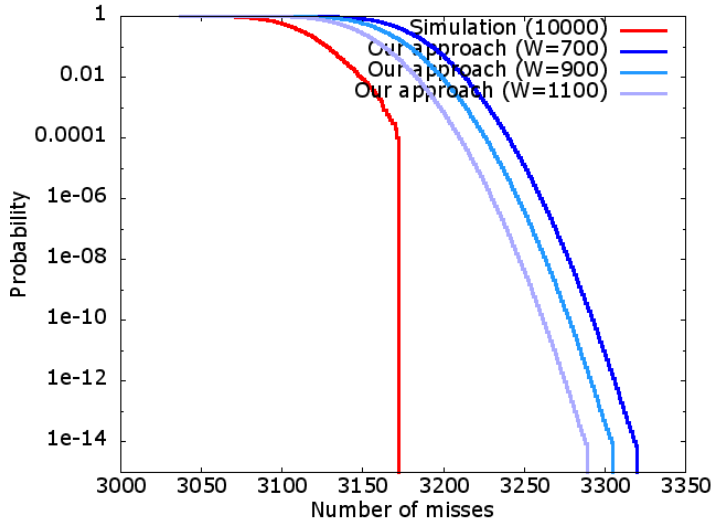


Figure 18: Impact of W on the precision of the analysis for one example benchmark *edn*.

From these curves, we can clearly see that the higher W is, the more precise is the result. This is the case because the bigger W is, the fewer windows appear in the analysis result. Moreover, fewer windows imply that fewer analyses are performed, and thus fewer cache reinitializations are made. Since the loss in precision comes from this reinitialization of the cache state for every analysis, increasing W causes a gain in precision.

However, we have seen in Figure 17 that increasing W also increases the execution time. Thus, a trade-off between execution time and precision must be made when performing an analysis.

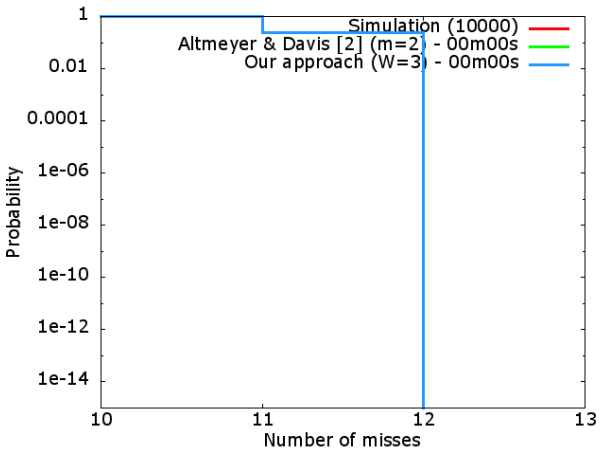
4.3 Results on the reference programs

We have seen before that the complexity of the analysis increases with W. Therefore, a trade-off must be made. For this reason, in the remaining of this document, we set W as the maximum value causing an execution time just below 15 minutes (or causing the windows to contain the independent parts if lasting less than 15 minutes). In order to compare to the state-of-the art results obtained with the approach of [2], we use a value of m that also causes an execution time just below one hour. Experimentally, we obtain the following values for the programs in Table 2:

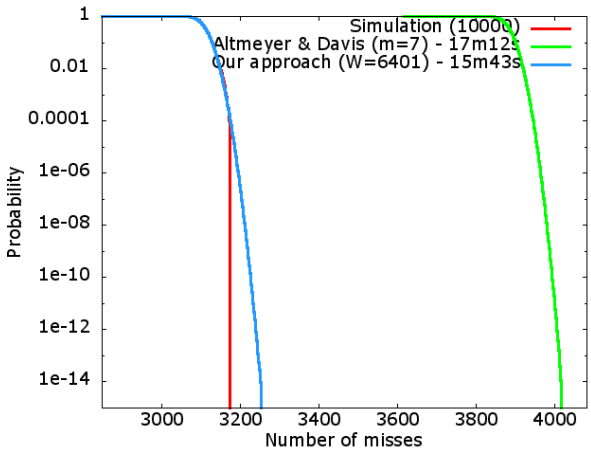
Name	Value for W	Computation time	Value for m	Computation time
bs	3 (max reached)	00m00s	2 (max reached)	00m00s
edn	6401 (max reached)	15m43s	7	17m12s
fdct	107 (max reached)	00m13s	14 (max reached)	00m13s
jfdctint	90 (max reached)	00m05s	12 (max reached)	00m04s
lcdnum	31 (max reached)	00m00s	3 (max reached)	00m00s
ludcmp	434 (max reached)	00m12s	13 (max reached)	00m31s
matmult	844 (max reached)	00m10s	4 (max reached)	00m05s
minver	20 (max reached)	00m01s	18 (max reached)	00m01s
nsichneu	20	13m49s	60	14m41s
sqrt	41 (max reached)	00m00s	3 (max reached)	00m00s

Table 3: Values for W and m that cause an execution to last just below 15 minutes. Parameters for which *(max reached)* is indicated correspond to those for which increasing the value will not change the execution time. This happens for W when the independent parts are fully separated into windows, and for m when all the blocks are studied exhaustively.

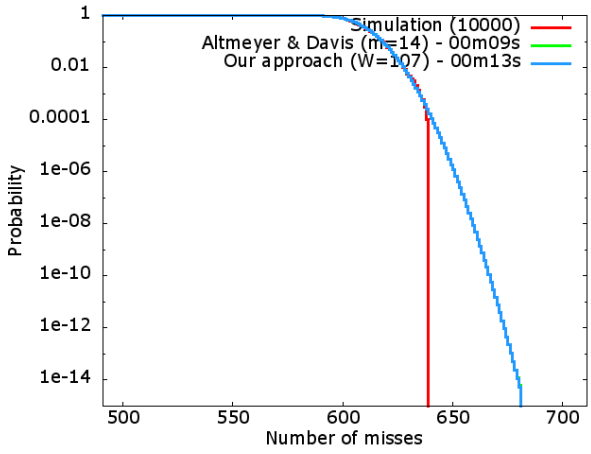
The curves in Figure 21 represent the CCDFs obtained using different analyses, for every program presented in Table 2, with the parameters indicated in Table 3. The red curve shows the distribution obtained when simulating the program a high number of times (here, 10000 runs). The blue one represents the result of our analysis using the third heuristic and the green one reprents the results from the approach in [2].



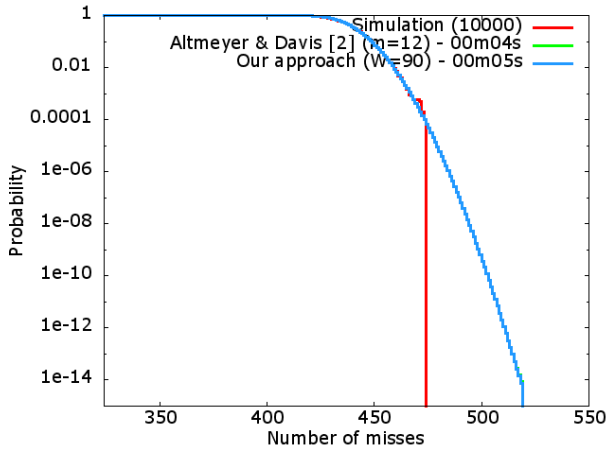
bs



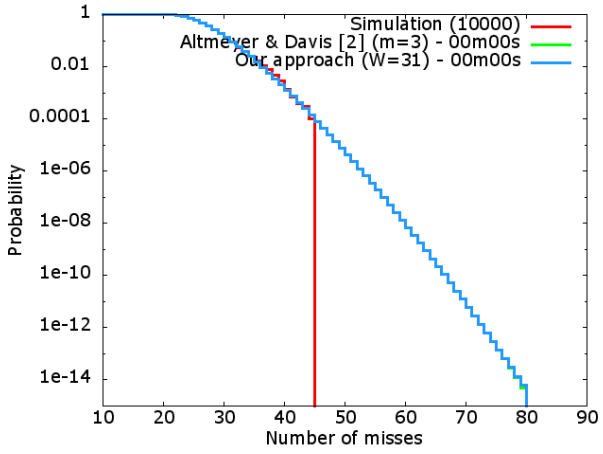
edn



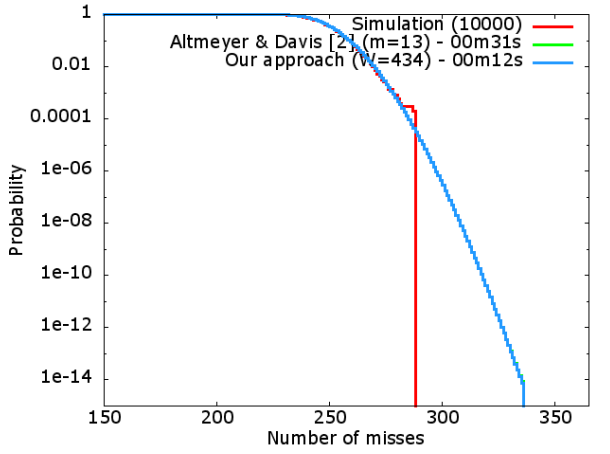
fdct



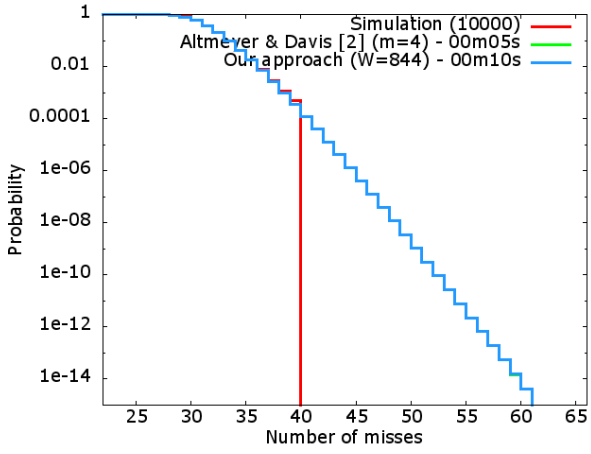
jfdctint



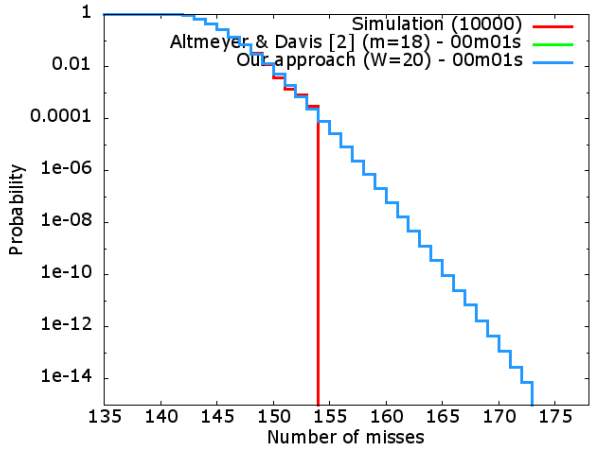
lcdnum



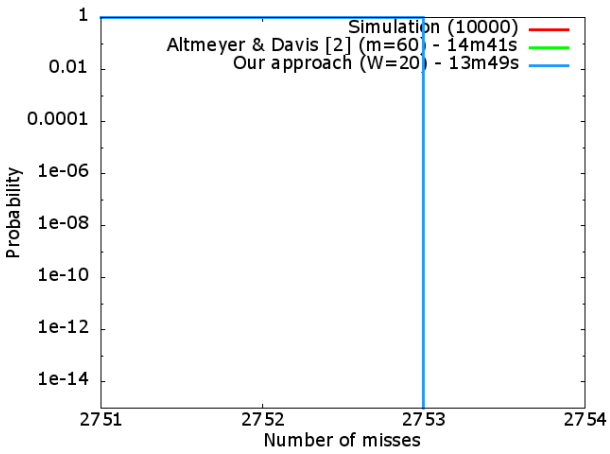
ludcmp



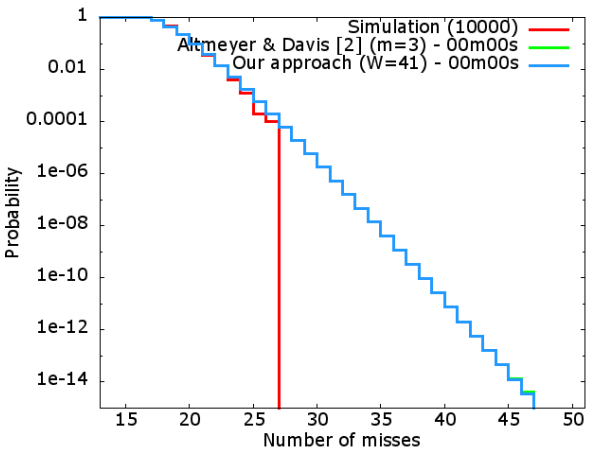
matmult



minver



nsichneu

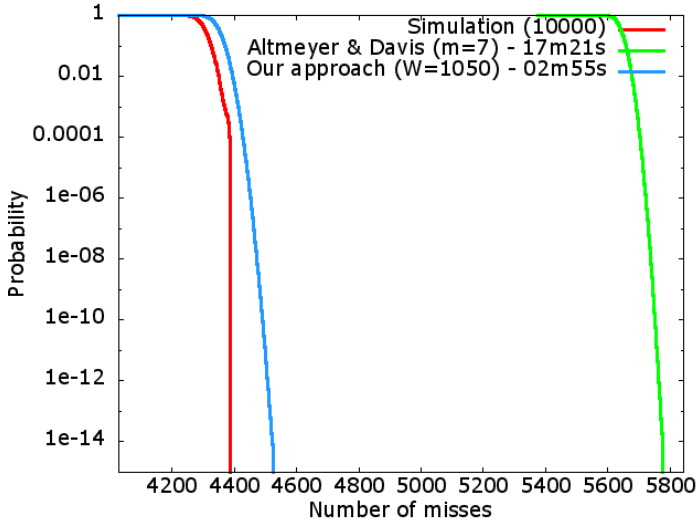


sqrt

Figure 21: Comparison of the precision of our approach compared to simulation and to the current state-of-the-art method. For most benchmarks, the results of the two analyses are mingled.

As it can be seen in the results of our evaluation, the precision of the two approaches is equivalent for small benchmarks. For bigger programs, our approach is more precise than the one in [2] for an equivalent computation time. This is the case for example for the program *edn* for which we can reach the precision of an exhaustive analysis whereas the method of [2] cannot.

To compare the precision loss of our method when it cannot put the independent portions of the program in distinct windows, we create a bigger program by putting end-to-end the 4 following benchmarks: *fdct*, *jfdctint*, *minver* and *edn*. The resulting CCDF is presented in Figure 22.



fdct-jfdctint-minver-edn

Figure 22: Comparison of the precision of our approach compared to simulation and to the current state-of-the-art method for an artificially created benchmark.

From this benchmark, we can see that our analysis can reach an interesting precision in a short amount of time. It is however not the case for the method in [2]. This difference is explained by the fact that our method captures the reuse of references and then loses precision if needed. Thus, even in case of a cache reinitialization, most of the reuse is captured. On the other side, the method of [2] performs an exhaustive enumeration only for the most occurring references. Therefore, for programs with numerous loops, only the m most occurring loops are considered for the precise evaluation.

Because of this difference in the way the two methods work, an interesting direction for future work would be to study a combination of the method. It is theoretically possible to limit the size of the windows with a method similar to ours, and to replace our exhaustive enumeration by the method used in [2]. However, for this research direction, new heuristics to limit the size of the windows should be developped (by limiting for example the number of occurrences of references per window).

4.4 Impact of the cache structure on the precision and execution time

An important parameter that was not yet discussed is the impact of the cache structure on the analysis. We did not have the time to perform the same analyses as in Section 4.3 for a different cache structure (typically 4 sets, 8 ways and 32B lines).

The expected result is that by increasing the number of ways, it will cause the number of possible cache states to be greater, since an eviction for a given reference will become less possible when a miss occurs (we recall that the probability to be evicted in case of a miss is equal to $1/N$, where N is the associativity of the cache). Moreover, by decreasing the number of sets, there will be fewer independent windows (the addresses mapping to different sets are independent). Thus, independent windows will be bigger and some cuts will be needed to bound their sizes, leading to precision loss.

5 Conclusion

5.1 Summary of the contribution

In this document, we have introduced a method to study single-path programs. Our method falls into the category of static probabilistic methods, and provides a pET distribution that captures the probabilities related to the randomized aspects of the hardware. It consists in three steps. First, a partition of the program is made. Then, an exhaustive cache states analysis is performed on every window in the partition. Finally, the results are combined using convolutions to provide a pET distribution for the whole program.

Because the analyses are performed independently assuming a worst-case initial situation for each of them, the produced pET distribution is safe by construction. Tightness of our method was experimentally evaluated by comparing to a great number of simulations, and to the current state-of-the-art static probabilistic method [2]. We have shown that our method has a precision that is equivalent to the results in [2] for small programs, and that provides interesting results for bigger programs.

5.2 Future work

Numerous extensions can be done with this work. The first of them is introducing parallelization. We have shown that the various windows are independently analyzed. Therefore, studying them can be done simultaneously. Moreover, the exhaustive cache states enumeration consists in exploring a tree. Thus, parallelization can also be introduced to consider multiple branches in parallel.

Another extension that must be done is studying more deeply the combination of our approach with the one in [2]. New heuristics should be developped to split the program into small parts in order to bound the number of memory blocks per window in a way that reduces the execution time and enhances the precision of the combined approach.

The most important task that is left is to allow the consideration of multi-path programs. A first idea would be to perform as in Section 2.2.3, by computing pET distributions for distinct sections in the program, and then to combine them using convolution/maximum.

References

[1] Jaume Abella, Damien Hardy, Isabelle Puaut, Eduardo Quinones, and Francisco J. Carzorla. On the comparison of deterministic and probabilistic WCET estimation techniques. In *26th euromicro conference on real-time systems (ECRTS)*. To appear, 2014.

[2] Sebastian Altmeyer and Robert I. Davis. On the correctness, optimality and precision of static probabilistic timing analysis. In *Proceedings of the conference on design, automation & test in europe*. European Design and Automation Association, 2014.

[3] Dale Berger. A gentle introduction to resampling techniques. Claremont Graduate University.

[4] Guillem Bernat, Antoine Colin, and Stefan M. Petters. WCET analysis of probabilistic hard real-time systems. In *Proceedings of the 23rd IEEE real-time systems symposium*. IEEE Computer Society, 2002.

[5] Francisco J. Cazorla, Eduardo Quiñones, Tullio Vardanega, Liliana Cucu, Benoit Triquet, Guillem Bernat, Emery Berger, Jaume Abella, Franck Wartel, Michael Houston, Luca Santinelli, Leonidas Kosmidis, Code Lo, and Dorin Maxim. Proartis: probabilistically analyzable real-time systems. *ACM trans. embed. comput. syst.*, 2013.

[6] Stuart Coles. *An introduction to statistical modeling of extreme values*. Springer, 2001.

[7] Antoine Colin and Isabelle Puaut. Worst case execution time analysis for a processor with branch prediction. *RTS*, 2000.

[8] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Conference record of the fourth annual ACM SIGPLAN-SIGACT symposium on principles of programming languages*. ACM, 1977.

[9] Liliana Cucu-Grosjean, Luca Santinelli, Michael Houston, Code Lo, Tullio Vardanega, Leonidas Kosmidis, Jaume Abella, Enrico Mezzetti, Eduard Quiñones, and Francisco J. Cazorla. Measurement-based probabilistic timing analysis for multi-path programs. In. Robert Davis, 2012.

[10] Laurent David and Isabelle Puaut. Static determination of probabilistic execution times. In *Proceedings of the 16th euromicro conference on real-time systems*. IEEE Computer Society, 2004.

[11] Robert I Davis, Luca Santinelli, Sebastian Altmeyer, Claire Maiza, and Liliana Cucu-Grosjean. Analysis of probabilistic cache related pre-emption delays. In *25th euromicro conference on real-time systems (ECRTS)*. IEEE, 2013.

[12] Jakob Engblom. Processor pipelines and static worst-case execution time analysis. PhD thesis. Uppsala University, 2002.

[13] Christian Ferdinand and Reinhard Wilhelm. Efficient and precise cache behavior prediction for real-time systems. *Real-time syst.*, 1999.

[14] Damien Hardy and Isabelle Puaut. WCET analysis of instruction cache hierarchies. *J. syst. archit.*, 2011.

[15] Leonidas Kosmidis, Jaume Abella, Eduardo Quiñones, and Francisco J. Cazorla. A cache design for probabilistically analysable real-time systems. In *Proceedings of the conference on design, automation and test in europe*. EDA Consortium, 2013.

[16] Leonidas Kosmidis, Luca Santinelli, Michael Houston, Liliana Cucu, Eduardo Quinones, Jaume Abella, Tullio Vardanega, and Francisco J Cazorla. A case study for a PTA-friendly processor. 2013.

[17] MR Leadbetter. Extremes and local dependence in stationary sequences. *Probability theory and related fields*, 1983.

[18] Yau-Tsun Steven Li and Sharad Malik. Performance analysis of embedded software using implicit path enumeration. In *Proceedings of the 32nd annual ACM/IEEE design automation conference*. ACM, 1995.

[19] Dorin Maxim. Analyse probabiliste des systèmes temps réel. PhD thesis. 2013.

[20] Dorin Maxim, Mike Houston, Luca Santinelli, Guillem Bernat, Robert I. Davis, and Liliana Cucu-Grosjean. Re-sampling for statistical timing analysis of real-time systems. In *Proceedings of the 20th international conference on real-time and network systems*. ACM, 2012.

[21] Alessandra Melani, Eric Noulard, and Luca Santinelli. Learning from probabilities: dependences within real-time systems. In *ETFA*, 2013.

[22] Peter Puschner and Ch. Koza. Calculating the maximum execution time of real-time programs. *RTS*, 1989.

[23] Jan Reineke, Daniel Grund, Christoph Berg, and Reinhard Wilhelm. Timing predictability of cache replacement policies. *Real-time syst.*, 2007.

[24] Alan Jay Smith. Cache memories. *ACM comput. surv.*, 1982.

[25] Ingomar Wenzel, Raimund Kirner, Peter Puschner, and Bernhard Rieder. Principles of timing anomalies in superscalar processors. In *Proceedings of the fifth international conference on quality software*. IEEE Computer Society, 2005.

[26] Reinhard Wilhelm, Jakob Engblom, Andreas Ermedahl, Niklas Holsti, Stephan Thesing, David Whalley, Guillem Bernat, Christian Ferdinand, Reinhold Heckmann, Tulika Mitra, Frank Mueller, Isabelle Puaut, Peter Puschner, Jan Staschulat, and Per Stenström. The worst-case execution-time problem – overview of methods and survey of tools. *ACM trans. embed. comput. syst.*, 2008.

[27] Nicky Williams, Bruno Marre, Patricia Mouy, and Muriel Roger. Pathcrawler: automatic generation of path tests by combining static and dynamic analysis. In *In: proc. european dependable computing conference. volume 3463 of LNCS (2005) 281–292*. Springer, 2005.

[28] Shuchang Zhou. An efficient simulation algorithm for cache of random replacement policy. In, *Network and parallel computing*. Springer Berlin Heidelberg, 2010.